MINI PROJECT REPORT

ON

"Library Management System with DevOps Automation and Observability"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Aman Raychaudhuri (T21 2401094)
Nikhil Rajani (T21 2401087)
Asmit Pandey (T21 2401072)

SUBJECT

ASD&D

SUPERVISOR

Prof. Sanobar Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050
UNIVERSITY OF MUMBAI
[Academic Year: 2025–26]



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled “Library Management System with DevOps Automation and Observability” is a bonafide work of Aman Raychaudhuri, Nikhil Rajani, Asmit Pandey submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of “Undergraduate” in “Artificial Intelligence and Data Science”.

_____

Prof. Sanobar Shaikh
(Supervisor)

Abstract

Small college and departmental libraries often keep their catalogue, membership and circulation records in registers or disconnected spreadsheets. This makes it slow to locate a title, uncertain whether a copy is on the shelf, and difficult to see the overall state of the collection. This project presents a Library Management System (LMS): a simple, browser-based administration interface that brings book records, member records and issue and return workflows together with a dashboard summary. The interface is built as a single-page application with React and the Vite build tool. Records are persisted in the browser's localStorage through a layer of frontend service functions; the current version intentionally has no backend server or central database.

Alongside the application, the project applies a DevOps workflow. Git and GitHub provide source control. A multi-stage Dockerfile compiles the production build and packages the static output into an Nginx-based image, and Kubernetes manifests describe a Deployment and a Service that run and expose that image on a local Docker Desktop cluster. The Grafana Faro Web SDK is integrated for browser-side observability where a collector URL is configured, so that JavaScript errors, Web Vitals and session information can be examined in Grafana Cloud.

Prometheus and Grafana metrics monitoring is examined in depth as a proposed extension. The report explains why a frontend-only application exposes no metrics endpoint that Prometheus could scrape, and it specifies the instrumented service or exporter, scrape configuration, PromQL queries and dashboard panels that a future version would require in order to collect and visualise service-level metrics. Every capability in the report is labelled with its implementation status, and a verification procedure is defined for each, so that results are reported only on the basis of observed evidence.

Keywords: React, Vite, localStorage, Docker, Nginx, Kubernetes, Grafana Faro, Prometheus, Grafana, DevOps, observability.

TABLE OF CONTENTS

Content	Page No.
Abstract	**3**
List of Figures	**5**
List of Tables	**6**
1. Introduction	**7**
1.1 Background	7
1.2 Problem Statement	7
1.3 Objectives	8
1.4 Scope of the Project	8
1.5 Limitations	10
2. Implementation	**11**
2.1 Technologies Used	11
2.2 System Architecture	12
2.3 Application Modules	14
2.4 Code Structure	16
2.5 Data Handling and Frontend Service Layer	17
2.6 Docker Containerization	19
2.7 Kubernetes Deployment and Orchestration	22
2.8 Frontend Observability Using Grafana Faro	26
2.9 Proposed Prometheus and Grafana Metrics Monitoring	30
2.10 Git and GitHub Workflow	37
2.11 Jira and Scrum Workflow	37
2.12 Continuous Integration with GitHub Actions or Jenkins	38
3. Results and Discussion	**40**
3.1 Application Functional Results	40
3.2 Docker Verification	41
3.3 Kubernetes Verification	42
3.4 Frontend Observability Results	42
3.5 Proposed Prometheus and Grafana Monitoring: Status and Design	43
3.6 Discussion	44
3.7 Limitations and Technical Considerations	44
Screenshots — Reserved Space	46
4. Conclusion and Future Scope	**51**
4.1 Conclusion	51
4.2 Future Enhancements	51
5. References	**53**
Appendices	**54**
Appendix A: Screenshot Checklist	54
Appendix B: Additional Test Cases	54
Appendix C: Important Commands	55

Content	Page No.
Appendix D: Sample Configuration Files	56

LIST OF FIGURES

Figure	Page No.
Figure 2.1: Existing frontend architecture: React UI, service functions and browser localStorage	12
Figure 2.2: Containerisation and deployment workflow (resource names as used in this report)	13
Figure 2.3: Issue and return workflow and its effect on availability	16
Figure 2.4: Multi-stage Docker build: Node.js build stage and Nginx runtime stage	20
Figure 2.5: Kubernetes objects for the LMS and the path of a request from the browser to a Pod	23
Figure 2.6: Grafana Faro telemetry flow from the browser to Grafana Cloud	27
Figure 2.7: Generic Prometheus and Grafana data flow (concept)	32
Figure 2.8: PROPOSED ARCHITECTURE: future Prometheus metrics flow for the LMS	33
Figure 2.9: PROPOSED ARCHITECTURE: Kubernetes layout for a future monitoring stack	34
Figure 3.1: Library dashboard with summary counts	46
Figure 3.2: Book management module	46
Figure 3.3: Member management module	46
Figure 3.4: Issue and return workflow	47
Figure 3.5: GitHub repository and commit history	47
Figure 3.6: Jira board	47
Figure 3.7: Successful Docker image build	48
Figure 3.8: Docker Desktop showing the running container	48
Figure 3.9: LMS opened at http://localhost:8080	48
Figure 3.10: Kubernetes node status	49
Figure 3.11: Kubernetes Deployment and Pods	49
Figure 3.12: Kubernetes Service	49
Figure 3.13: LMS opened through Kubernetes	50
Figure 3.14: Grafana Frontend Observability with received telemetry	50
Figure 3.15: Successful CI pipeline run	50

LIST OF TABLES

Table	Page No.
Table 1.1: Implementation status definitions	9
Table 1.2: Implementation status register	9
Table 2.1: Technology stack summary	11
Table 2.2: Book record fields	14
Table 2.3: Member record fields	15
Table 2.4: Characteristics of localStorage and their implications	17
Table 2.5: Difference between a Dockerfile, an image and a container	19
Table 2.6: Docker commands used in this project	22
Table 2.7: Kubernetes concepts used in this project	22
Table 2.8: Distinguishing the stages of a Kubernetes deployment	25
Table 2.9: Signals collected by the Faro Web SDK	27
Table 2.10: Proposed Faro custom events	29
Table 2.11: Three distinct monitoring situations	30
Table 2.12: Prometheus terminology	30
Table 2.13: Prometheus metric types	31
Table 2.14: Illustrative metrics for a future backend (not present in the current application)	33
Table 2.15: Proposed dashboard panels and the instrumentation each requires	35
Table 2.16: Comparison of Grafana Faro and Prometheus	36
Table 2.17: Git commands used in the workflow	37
Table 2.18: Proposed epic structure for the LMS backlog	38
Table 2.19: Stages of CI evidence	38
Table 3.1: Functional test cases for the LMS application	40
Table 3.2: Docker verification	41
Table 3.3: Kubernetes verification	42
Table 3.4: Grafana Faro verification	43
Table 3.5: Prometheus and Grafana implementation status	43
Table 4.1: Completed work and future work	51
Table A.1: Screenshot checklist	54
Table B.1: Edge-case test cases	55
Table C.1: Command reference	55

Chapter 1
Introduction

1.1 Background

The day-to-day work of a library rests on three connected sets of records: the catalogue of books, the register of members, and the circulation log of issues and returns. In many small college, departmental and community libraries these records are still maintained in paper registers or in spreadsheets kept by different people. While the collection is small this is manageable, but as it grows the records drift apart. A book entered as issued in the circulation register may still appear as available in the catalogue sheet, member details may be duplicated or left outdated, and answering a simple question such as "how many books are currently issued?" requires counting entries by hand.

The Library Management System (LMS) developed in this project addresses these difficulties with a single browser-based administration interface. The application is built with **React**, a component-based JavaScript library for user interfaces, and **Vite**, a build tool that provides a fast development server and an optimised production build. Records are persisted in the browser's **localStorage** through a small set of frontend service functions, so the application runs entirely in the browser without a backend server or database. This is a deliberate scope decision for a mini project: it keeps attention on the core library workflows and on the delivery pipeline around them.

The project is also intended as a practical exercise in modern software delivery. The source code is version-controlled with **Git** and hosted on **GitHub**. The production build is packaged into a **Docker** image served by **Nginx**, and that image is described declaratively in **Kubernetes** manifests for a local cluster. **Grafana Faro** is integrated to observe the application from the user's browser. **Prometheus** and **Grafana**, the most widely used open-source pair for server-side metrics monitoring, are studied in detail and presented as a planned extension, together with a clear account of what the current architecture would need before such monitoring becomes possible.

1.2 Problem Statement

Manual or spreadsheet-based library administration creates the following practical problems:

1. **Finding books quickly.** Searching a register, or scrolling through a spreadsheet by title or author, is slow and error-prone, particularly when spelling and formatting vary between entries.
2. **Tracking availability.** Whether a copy is on the shelf depends on cross-checking the catalogue against the circulation register. If either is not updated, the two disagree.
3. **Managing member records.** Member details may be recorded inconsistently, duplicated or left outdated, which makes it difficult to know who is registered to borrow.

4. **Recording issue and return activity.** Each transaction must be written in one place and reflected in another. A missed update leaves a book shown as issued after it has been returned, or the reverse.
5. **Keeping information consistent.** When several people maintain separate files there is no single source of truth for the current state of the library.
6. **Reviewing current status.** Summary figures such as total books, available books, issued books and registered members must be compiled manually whenever they are needed.

The problem addressed by this project is therefore to provide a single, easy-to-use interface in which a librarian can maintain book and member records, issue and return books with availability updated automatically, and view the current status of the library on a dashboard; and to build, package, deploy and observe that interface using reproducible DevOps practices appropriate to an undergraduate project. The project accordingly demonstrates a complete software-development lifecycle: planning and task management, version control, containerisation, orchestration, observability, and build automation where it has actually been configured.

1.3 Objectives

1. To design and implement a React user interface for managing books, members, and issue and return transactions.
2. To organise all data operations into frontend service functions that read from and write to browser localStorage, so that records persist across page refreshes in the same browser profile.
3. To provide a dashboard that summarises the current state of the library, such as total books, available books, issued books and registered members, as implemented in the application.
4. To maintain the source code in Git with a remote repository on GitHub, using meaningful, incremental commits.
5. To containerise the production build with a multi-stage Dockerfile and Nginx, and run it as a container with a mapped host port.
6. To describe the application declaratively with Kubernetes Deployment and Service manifests and run it on a local Kubernetes cluster.
7. To integrate the Grafana Faro Web SDK for browser-side error, performance and session telemetry where a collector URL is configured.
8. To study Prometheus and Grafana metrics monitoring in depth and design a proposed extension, covering the required metrics source, scrape configuration, PromQL queries and dashboard panels, without presenting it as implemented.
9. To define a verification procedure for every component so that each reported result is supported by observable evidence.

1.4 Scope of the Project

The scope of the project is divided into three layers:

- **Application scope.** A single-user, browser-based administration interface for books, members, issue and return, and a dashboard summary. Data is stored per browser profile in localStorage. User accounts, roles and server-side persistence are outside the current scope.
- **Delivery scope.** Version control with Git and GitHub, container packaging with Docker and Nginx, and local orchestration with Kubernetes on Docker Desktop. Public cloud hosting, a container registry and a production domain are outside the current scope.
- **Observability scope.** Browser-side telemetry through Grafana Faro, where configured. Prometheus and Grafana metrics monitoring is in scope only as a design study and proposed extension.

To keep the report accurate, every feature is assigned one of the four implementation statuses defined in Table 1.1. Table 1.2 records the status of each component on the basis of the evidence available when this report was prepared. A status is upgraded only when the corresponding screenshot or terminal output from Chapter 3 is attached.

Table 1.1: Implementation status definitions

Status	Meaning	Evidence required to claim it
Implemented and verified	The feature exists in the source code and has been observed working.	Screenshot or terminal output included in Chapter 3.
Implemented but not yet verified	The feature exists in the source code, but its working behaviour has not yet been documented.	Run the test procedure in Chapter 3 and attach the evidence.
Configured but not yet executed	Configuration exists (Dockerfile, YAML manifests, environment variables), but the build, deployment or data collection has not been demonstrated.	Execute the commands and record the actual output.
Proposed architecture or future enhancement	Designed or described only; not present in the current repository.	Implementation in a future version of the project.

Table 1.2: Implementation status register

Component	Description	Status
Dashboard	Summary counts derived from the stored records	Implemented but not yet verified
Book management	Add, list and delete book records	Implemented but not yet verified
Member management	Add, list and delete member records	Implemented but not yet verified
Issue and return	Circulation workflow with availability updates	Implemented but not yet verified

Component	Description	Status
localStorage service layer	Reading and writing JSON records in browser storage	Implemented but not yet verified
Git and GitHub	Version history and remote repository	Implemented but not yet verified
Docker image and container	Multi-stage Dockerfile, Nginx runtime, host port 8080 mapped to container port 80	Configured but not yet executed
Kubernetes Deployment and Service	Manifests in the k8s/ folder for the local Docker Desktop cluster	Configured but not yet executed
Grafana Faro integration	Web SDK initialised from VITE_GRAFANA_* variables	Configured but not yet executed
Faro custom events	book_added, book_issued and related events	Proposed architecture or future enhancement
Prometheus and Grafana metrics	Metrics endpoint, scraping and dashboards	Proposed architecture or future enhancement
Jira Scrum board	Backlog and sprint tracking	Proposed (update if Jira was used)
CI pipeline	GitHub Actions or Jenkins build checks	Proposed (update if a pipeline was run)

> **Note:** The descriptions of the application modules in Chapter 2 follow the workflows the application was built to support. Field names, file names and Kubernetes resource names that must match the repository exactly are marked with a highlighted *[Verify: ...]* note. Remove each note once the value has been checked against the source code.

1.5 Limitations

The following limitations follow directly from the chosen architecture and are acknowledged at the outset:

- **Data is local to one browser profile.** Records stored in localStorage are visible only to the same origin in the same browser profile on the same device. Clearing site data, using a private window, or switching to another browser or device presents an empty dataset.
- **No multi-user synchronisation.** Two librarians working on different computers see two independent datasets; there is no shared source of truth.
- **No authentication or authorisation.** Anyone with access to the browser can view or modify the records, including through the browser's developer tools.
- **Storage constraints.** localStorage stores strings only, is accessed synchronously on the main thread, and has a browser-dependent size quota of a few megabytes per origin.
- **No server-side validation, backup or audit trail.** Data integrity depends entirely on the frontend code.

- **Local orchestration only.** Kubernetes runs on a single-node Docker Desktop cluster. Running several replicas demonstrates scaling and self-healing of the web server, but it does not create shared data: each user's data still lives in that user's browser.
- **Prometheus monitoring is not implemented.** The current application has no metrics endpoint that Prometheus could scrape (Section 2.9).
- **Faro telemetry depends on configuration and network access.** It requires a valid collector URL at build time and can be blocked by browser privacy extensions.

Chapter 2
Implementation

2.1 Technologies Used

Frontend — React builds the user interface from reusable function components. Component state, managed with React hooks, holds the in-memory copy of the records that are currently displayed. The interface is written in JavaScript with JSX syntax and styled with CSS. *[Verify: confirm JavaScript (.jsx) or TypeScript (.tsx) from the repository]*

Build tooling — Vite provides the development server with hot module replacement (`npm run dev`) and the production build (`npm run build`), which writes minified, content-hashed static assets to the `dist/` folder.

Persistence — The Web Storage API's `localStorage` object provides key–value string storage scoped to the page origin. Data survives page reloads and browser restarts until it is cleared.

Version control — Git records the history of the source code; GitHub hosts the remote repository.

Containerisation — Docker and Docker Desktop build and run the container image. Nginx, a lightweight web server, serves the static production build inside the container.

Orchestration — Kubernetes, using the single-node cluster built into Docker Desktop, runs the image through a Deployment and exposes it through a Service. The `kubectl` command-line tool applies manifests and inspects resources.

Frontend observability — The Grafana Faro Web SDK (`@grafana/faro-web-sdk`) collects browser telemetry and sends it to Grafana Cloud Frontend Observability, where configured.

Metrics monitoring (proposed) — Prometheus would collect numeric time-series metrics from an instrumented service, and Grafana would visualise them in dashboards. This layer is a design study in the current project.

Project management and CI (proposed unless used) — Jira for Scrum backlog and sprint tracking; GitHub Actions or Jenkins for automated build checks.

Table 2.1: Technology stack summary

Layer	Technology	Role in this project	Status
User interface	React, JSX, CSS	Components for dashboard, books, members, issue/return	Implemented, not yet verified
Build	Vite, npm	Development server and production bundle	Implemented, not yet verified
Persistence	localStorage (Web Storage API)	Browser-side storage of JSON records	Implemented, not yet verified

Layer	Technology	Role in this project	Status
Source control	Git, GitHub	History, branches, remote repository	Implemented, not yet verified
Packaging	Docker, Nginx	Multi-stage image serving static files	Configured, not yet executed
Orchestration	Kubernetes, kubectl	Deployment, Pods, Service on Docker Desktop	Configured, not yet executed
Browser telemetry	Grafana Faro, Grafana Cloud	Errors, Web Vitals, sessions	Configured, not yet executed
Metrics monitoring	Prometheus, PromQL, Grafana	Service metrics and dashboards	Proposed
Process and CI	Jira, GitHub Actions or Jenkins	Planning and automated builds	Proposed unless used

2.2 System Architecture

The system has two complementary views. The **runtime view** (Figure 2.1) shows how the application behaves inside a user's browser. The **delivery view** (Figure 2.2) shows how the source code becomes a running, reachable application through Docker and Kubernetes.

2.2.1 Existing Frontend Architecture

The application follows a three-layer arrangement that runs entirely in the browser. React components form the presentation layer and never touch storage directly. Instead, every read and write goes through the frontend service functions, which serialise records to JSON and store them in localStorage. When the Faro SDK is configured, it runs alongside the application and sends browser telemetry to Grafana Cloud. This path is separate from the data path and carries no library records.

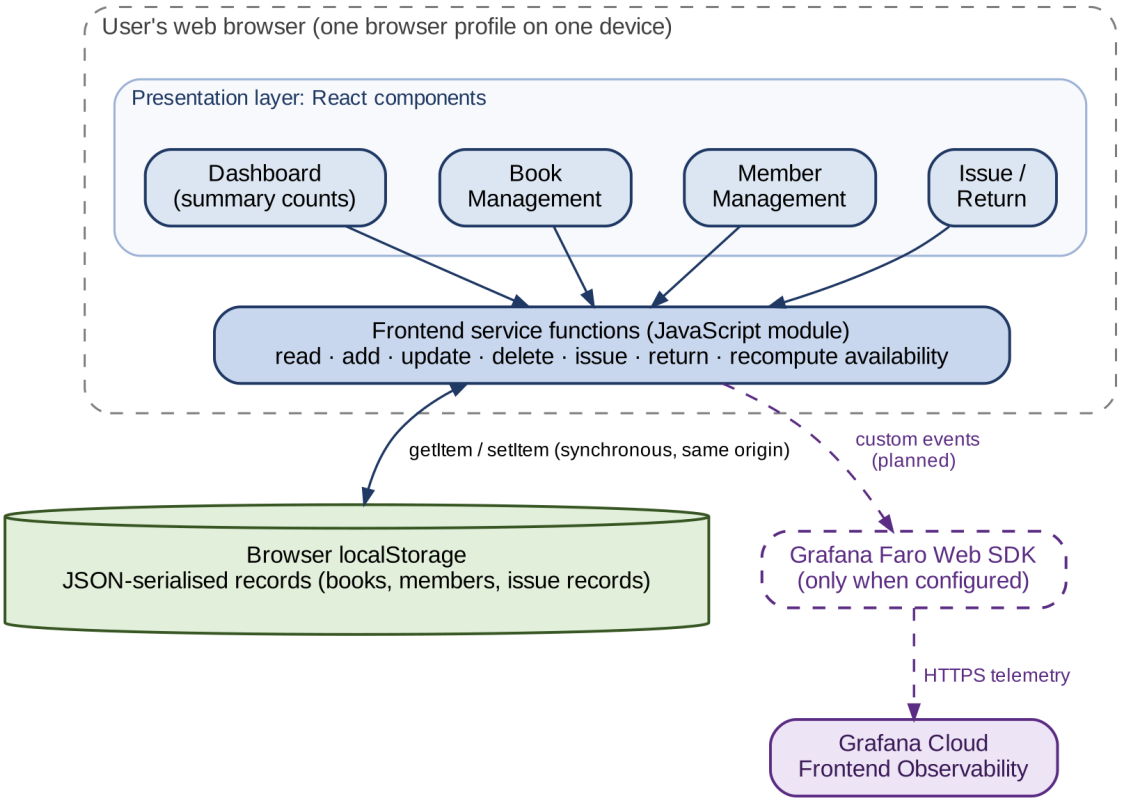


Figure 2.1: Existing frontend architecture: React UI, service functions and browser localStorage

This separation has a practical benefit. Because components call functions such as `getBooks()` or `issueBook()` rather than `localStorage` directly, the storage mechanism is isolated in one module. A future version could replace the bodies of those functions with HTTP calls to a backend API without rewriting the components.

2.2.2 Containerisation and Deployment Architecture

Figure 2.2 shows the intended delivery workflow. The React and Vite source code is compiled inside a multi-stage Docker build, producing an image that contains only Nginx and the static build output. The same image can then be run directly as a Docker container (Path A), or deployed to Kubernetes, where a Deployment maintains the desired number of Pods and a Service gives them a stable network endpoint (Path B).

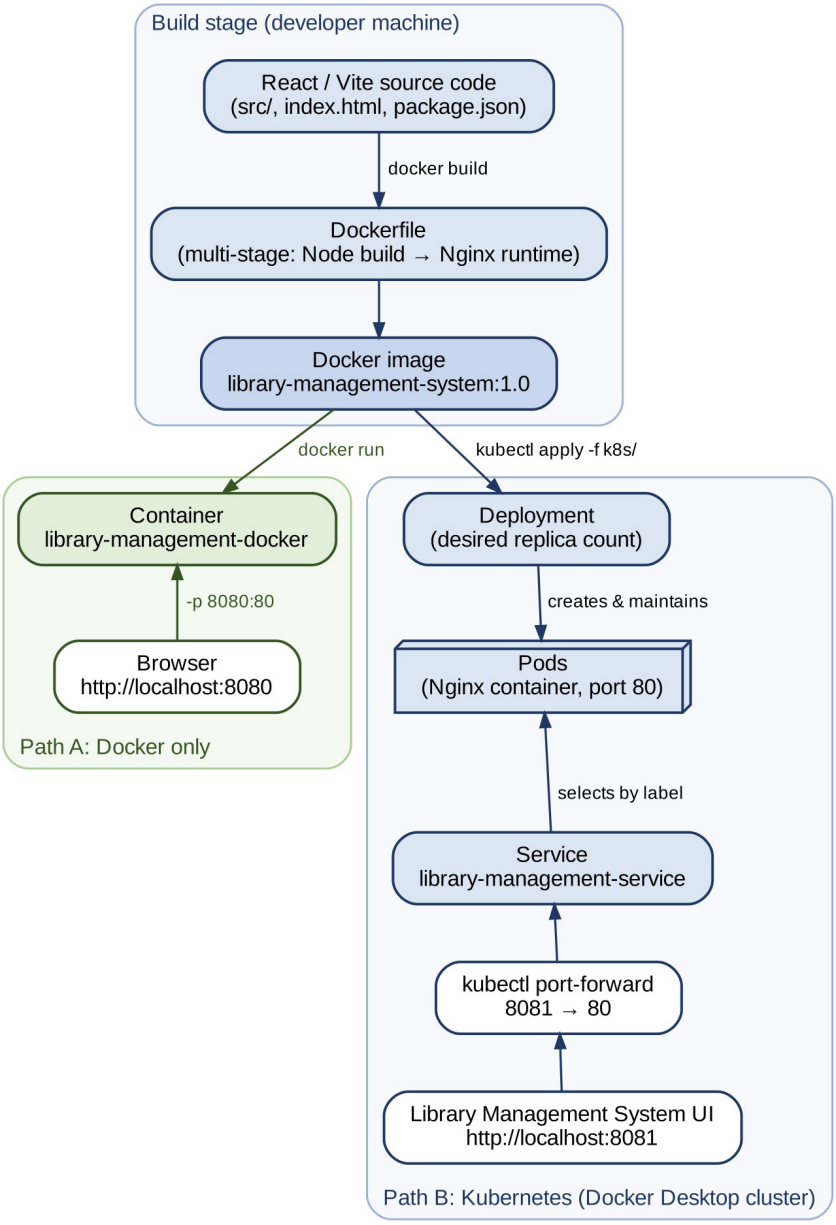


Figure 2.2: Containerisation and deployment workflow (resource names as used in this report)

> **Important:** localStorage is scoped to the page **origin**, which is the combination of scheme, host and port. The application opened at http://localhost:5173 (Vite dev server), http://localhost:8080 (Docker) and http://localhost:8081 (Kubernetes port-forward) therefore has three separate, independent datasets. This is expected behaviour, not data loss, and it is a useful point to explain during the demonstration.

2.3 Application Modules

The application is organised into four functional modules, each implemented as a React view that uses the shared service layer.

2.3.1 Dashboard

Implementation status: Implemented but not yet verified — attach Figure 3.1 to verify.

The dashboard is the landing view and presents summary figures for the library, such as total books, available books, issued books and registered members. *[Verify: keep only the summary cards actually shown on the dashboard]* The figures are derived from the stored records each time the view renders rather than being stored separately. For example, the number of issued books is computed from the active issue records or availability flags. Because the counts are recalculated from the same data the other modules modify, the dashboard cannot drift out of step with the underlying records, which is precisely the inconsistency described in Section 1.2.

2.3.2 Book Management

Implementation status: Implemented but not yet verified — attach Figure 3.2 to verify.

The book module maintains the catalogue. A librarian enters book details through a form; the record is validated, assigned a unique identifier, and appended to the stored list of books. Existing books are displayed in a list or table, and a book can be deleted when it is no longer part of the collection. Table 2.2 lists the fields used by the module.

Table 2.2: Book record fields

Field	Purpose
Book ID	Unique identifier generated by the application when the record is created
Title	Name of the book; the main field used to identify and find a book
Author	Author or authors of the book
Category / genre (if present)	Subject grouping used for browsing
ISBN (if present)	Standard book number for unambiguous identification
Availability or copy count	Whether the book (or how many copies) can currently be issued; changed by the issue and return workflow

[Verify: replace Table 2.2 with the exact field names used in the book form and stored records]
Supported operations are adding, listing and deleting books. A recommended safeguard is to prevent deletion of a book that is currently issued, because doing so would leave an issue record that refers to a book that no longer exists. Whether this rule is enforced should be confirmed during testing (Appendix B, test EC-04).

2.3.3 Member Management

Implementation status: Implemented but not yet verified — attach Figure 3.3 to verify.

The member module maintains the register of people who may borrow books. Its structure mirrors the book module: a form to add a member, a list of registered members, and an option to delete a member record.

Table 2.3: Member record fields

Field	Purpose
Member ID	Unique identifier generated by the application
Name	Full name of the member
Contact details (e.g., email, phone)	Used to reach the member about issued books
Department / class (if present)	Academic grouping of the member

[Verify: replace Table 2.3 with the exact member fields used in the repository] Member records contain personal information. In the current design this information never leaves the browser through the application itself, and it must not be included in telemetry events or metric labels (Sections 2.8 and 2.9).

2.3.4 Issue and Return

> **Implementation status: Implemented but not yet verified** — attach Figure 3.4 to verify.

The circulation module links books and members. To issue a book, the librarian selects a book and a member. The application checks that the book is available; if it is, an issue record is created containing the book identifier, the member identifier and the issue date, and the book is marked unavailable (or its available-copy count is reduced). If the book is not available, the issue is blocked and a message is shown. To return a book, the librarian selects an active issue record; the record is marked returned with a return date, and the book becomes available again. Both operations write the updated arrays back to localStorage, after which React re-renders and the dashboard counts change accordingly (Figure 2.3).

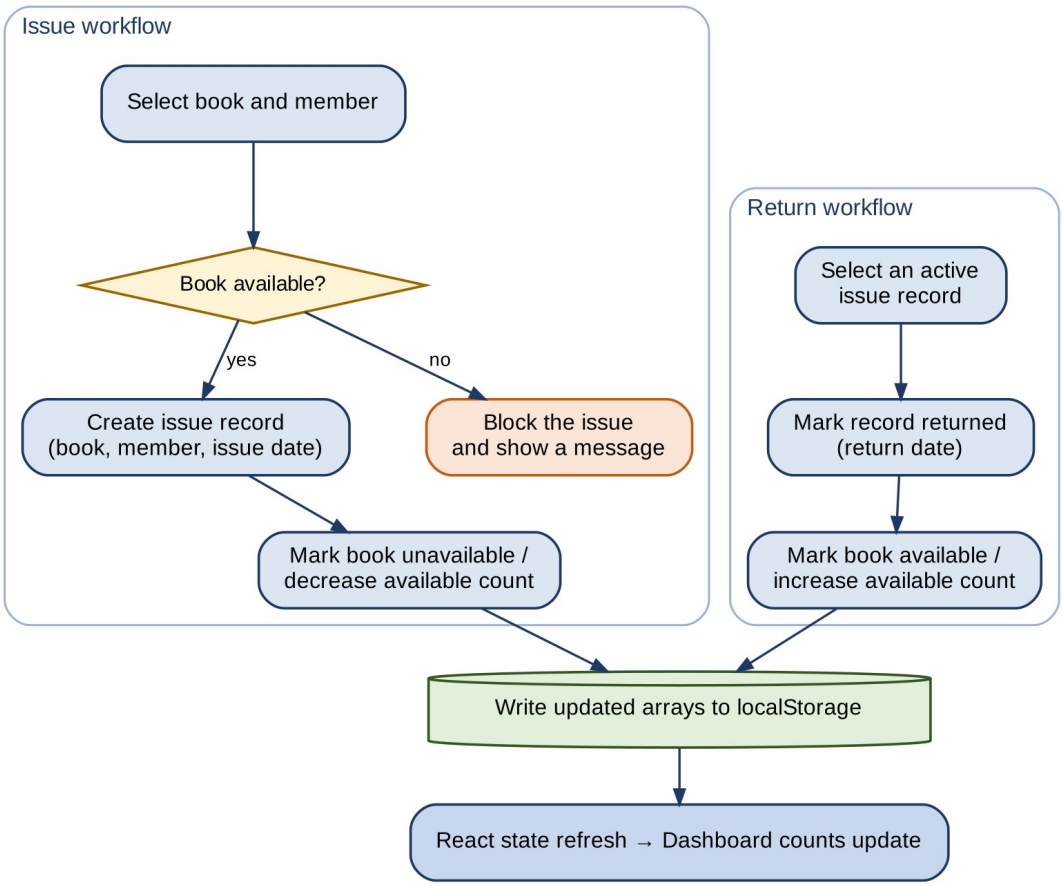


Figure 2.3: Issue and return workflow and its effect on availability

Three edge cases deserve attention during testing: issuing a book that is already issued, returning the same issue record twice, and deleting a member who still has a book issued. Each is listed as a test case in Chapter 3 and Appendix B so that the actual behaviour can be recorded rather than assumed.

2.4 Code Structure

The repository follows the standard layout of a Vite React project, with deployment configuration kept beside the application code so that each part can be developed and reviewed independently. *[Verify: match each path below to the actual repository tree, e.g. from the output of git ls-files]*

- `index.html` — Vite entry page that loads the application bundle.
- `src/main.jsx` — application bootstrap; mounts the React root and is the natural place to initialise Faro.
- `src/App.jsx` — top-level layout and navigation between the dashboard, books, members and issue/return views.
- `src/components/` or `src/pages/` — React components for each module.
- `src/services/` — frontend service functions that read and write localStorage.
- `src/*.css` — application styles.

- `package.json` — dependencies and the `dev`, `build` and `preview` scripts.
- `vite.config.js` — Vite configuration.
- `Dockerfile`, `.dockerignore`, `nginx.conf` — container build and web-server configuration.
- `k8s/` — Kubernetes manifests, for example `deployment.yaml` and `service.yaml`.
- `.env.example` — names of the `VITE_GRAFANA_*` variables, with no real values.

2.5 Data Handling and Frontend Service Layer

> **Implementation status: Implemented but not yet verified** — confirm by inspecting the browser's Local Storage panel (test TC-12).

2.5.1 How localStorage Works

localStorage is part of the Web Storage API implemented by every modern browser. It exposes a small synchronous interface: `setItem(key, value)` stores a string, `getItem(key)` returns the string or `null` if the key does not exist, `removeItem(key)` deletes one entry and `clear()` deletes all entries for the origin. Because only strings can be stored, arrays of records are converted with `JSON.stringify` before writing and restored with `JSON.parse` after reading. Data is kept per origin and per browser profile, and it remains until the user or the application clears it. This is why records survive a page refresh or a browser restart on the same machine.

Table 2.4: Characteristics of localStorage and their implications

Property	Behaviour	Implication for the LMS
Scope	Per origin (scheme + host + port) and per browser profile	Different ports or browsers show different datasets
Data type	Strings only	Records must be serialised as JSON
Access	Synchronous, on the main thread	Fast for small data; large datasets could slow the UI
Capacity	Browser-dependent quota, typically a few MB	Adequate for a demonstration-scale catalogue
Lifetime	Persists until cleared	Survives refreshes; lost when site data is cleared
Security	Readable and editable by any script on the origin and through developer tools	Not suitable as a secure store for personal data
Sharing	No server, no synchronisation	Each user has an independent copy of the data

2.5.2 The Service-Function Pattern

The service layer centralises every data operation. Listing 2.1 shows the pattern in a simplified form; the repository's own service module is the authoritative implementation, and its key names and function names may differ.

Listing 2.1: Representative frontend service functions (simplified pattern)

```
const KEYS = { books: 'lms_books', members: 'lms_members', issues: 'lms_issues' };

function read(key) {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : [];
  } catch {
    return []; // missing, corrupted or blocked storage: start with an empty list
  }
}

function write(key, records) {
  localStorage.setItem(key, JSON.stringify(records));
}

export const getBooks = () => read(KEYS.books);

export function addBook(book) {
  const record = { ...book, id: crypto.randomUUID(), available: true };
  write(KEYS.books, [...getBooks(), record]);
  return record;
}

export function issueBook(bookId, memberId) {
  const books = getBooks();
  const book = books.find((b) => b.id === bookId);
  if (!book || !book.available) throw new Error('Book is not available');
  const updated = books.map((b) => (b.id === bookId ? { ...b, available: false } :
b));
  write(KEYS.books, updated);
  write(KEYS.issues, [...read(KEYS.issues), {
    id: crypto.randomUUID(), bookId, memberId,
    issuedOn: new Date().toISOString(), returnedOn: null,
  }]);
}
```

Three design points are visible in this pattern. First, reading is defensive: if the stored value is missing or cannot be parsed, the function returns an empty list instead of crashing the interface. Second, every write replaces the whole array, which keeps the logic simple at the cost of rewriting the full list on each change. This is acceptable at demonstration scale. Third, business rules such as “a book must be available before it is issued” live in the service layer, so every component that issues a book obeys the same rule.

2.5.3 Why These Functions Are Not REST APIs

Although functions such as `getBooks()` and `addBook()` resemble the operations of an API, they are not REST APIs. A REST API is an interface exposed by a server over HTTP: clients send requests such as `GET /api/books` across a network, the server applies the operation to a shared data store, and it replies with a status code and a response body. The LMS service functions, by contrast, are ordinary JavaScript functions executed by the browser’s JavaScript engine in the same page as the components that call them. No HTTP request is made, no network is involved, no status codes are returned, and no other client can call them. Describing them as an API would misrepresent the architecture. Their API-like shape is nevertheless deliberate, because it is what allows a future backend to be introduced behind the same function signatures.

2.5.4 Why localStorage Provides No Multi-User Synchronisation or Server-Side Security

localStorage lives inside one browser profile on one device. There is no central copy of the data, so changes made by one librarian are never seen by another, and there is no mechanism to resolve conflicting edits. Even two tabs of the same browser write to the same storage without coordination, so the last write wins unless the application listens for the browser's `storage` event. There is also no server to enforce access control: any person or script with access to the origin can read or modify the stored values, and the data is not encrypted by the application. A cross-site scripting flaw would expose everything in storage. These properties are acceptable for a single-user demonstration, but a real multi-user library system would require a backend with authentication, authorisation, validation and a central database (Section 4.2).

2.6 Docker Containerization

> **Implementation status: Configured but not yet executed** — the Dockerfile exists; build and run evidence is recorded in Section 3.2.

2.6.1 What Docker Is and Why Containerisation Is Useful

Docker is a platform for packaging an application together with everything it needs to run, including the runtime, system libraries and configuration, into a portable unit called a container image. A container is an isolated process started from that image. Unlike a virtual machine, a container shares the host operating system kernel, so it starts in seconds and uses few resources.

For this project, containerisation removes the classic "it works on my machine" problem. The examiner, a team member or a Kubernetes cluster can run exactly the same artefact without installing Node.js, without running `npm install`, and without depending on a particular operating-system configuration. The image also becomes the single unit that later stages, Kubernetes in particular, deploy and manage.

2.6.2 Dockerfile, Image and Container

Table 2.5: Difference between a Dockerfile, an image and a container

Concept	What it is	In this project
Dockerfile	A text file of instructions that describes how to build an image	The Dockerfile at the repository root (Listing 2.2)
Image	A read-only, layered template produced by `docker build`; identified by a name and tag	`library-management-system:1.0`
Container	A running (or stopped) instance of an image, with its own process, filesystem layer and network settings	`library-management-docker`, started with `docker run`

The relationship is similar to that between source code, a compiled program and a running process: one Dockerfile builds one image, and one image can start many containers.

2.6.3 Why the Production Build Is Created Before Deployment

The Vite development server (`npm run dev`) is designed for development. It serves source modules on demand, injects hot-reload code and does not minify output. It is neither efficient nor intended for serving users. `npm run build` instead compiles the JSX, bundles and minifies the JavaScript and CSS, adds content hashes to file names for cache control, and writes plain static files to `dist/`. Static files need no Node.js process at run time and can be served by any web server, which keeps the runtime container small and simple.

2.6.4 Role of Nginx

Nginx is a lightweight, high-performance web server. In the runtime image it serves the files in `dist/` on port 80. A single-page application needs one additional rule: when the browser requests a client-side route directly (for example after a refresh on `/books`), there is no matching file on disk, so Nginx must fall back to `index.html` and let React render the route. This is achieved with `try_files` (Listing 2.3). Without it, refreshing on any route other than `/` would return HTTP 404. Nginx holds no library data; it only delivers the application code to the browser.

2.6.5 Multi-Stage Build

A multi-stage Dockerfile uses several `FROM` instructions. The first stage uses a Node.js image to install dependencies and run the production build; the second stage starts again from a small Nginx image and copies in only the `dist/` folder. Everything else (`node_modules`, the source code and the build tools) is left behind in the discarded build stage, so the final image is smaller, has fewer packages that could contain vulnerabilities, and contains nothing except what is needed to serve the application (Figure 2.4).

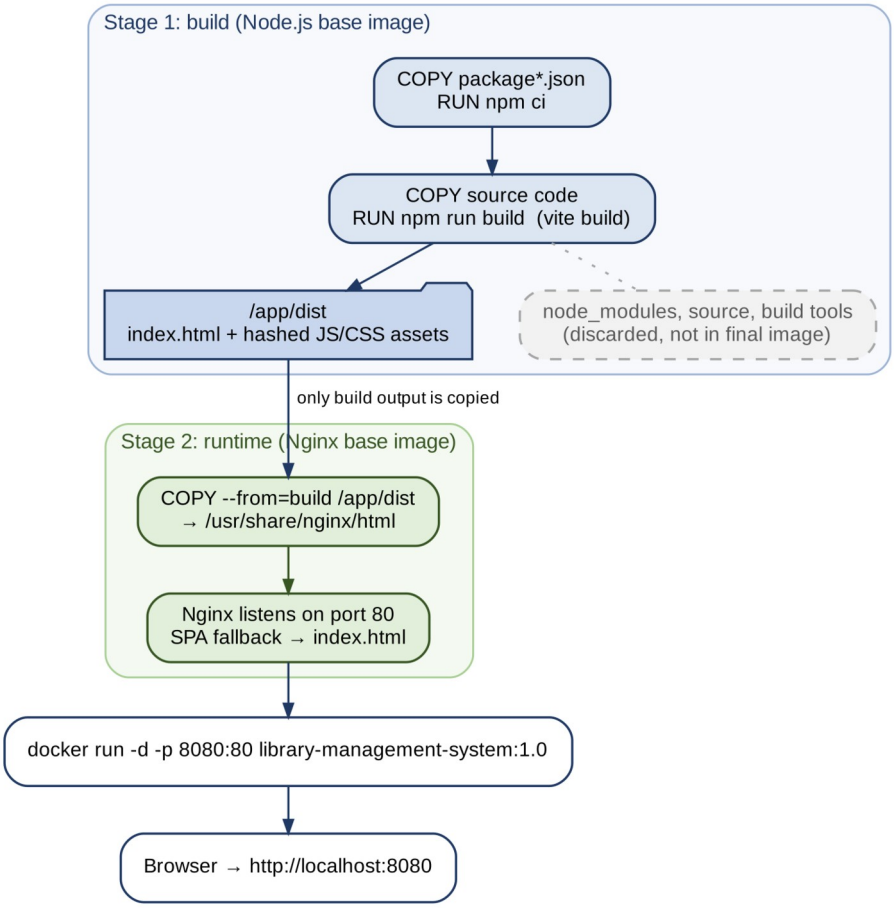


Figure 2.4: Multi-stage Docker build: Node.js build stage and Nginx runtime stage

Listing 2.2: Representative multi-stage Dockerfile (the repository file is authoritative)

```
# ---------- Stage 1: build the production bundle ----------
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
# VITE_* values are embedded at build time (see Section 2.8.5)
ARG VITE_GRAFANA_FARO_URL
ARG VITE_GRAFANA_APP_NAME=library-management-system
ARG VITE_GRAFANA_APP_ENV=development
RUN npm run build

# ---------- Stage 2: serve the static files with Nginx ----------
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 2.3: Nginx configuration with single-page-application fallback

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Copying `package*.json` and running `npm ci` before copying the rest of the source makes good use of Docker's layer cache: when only application code changes, the dependency layer is reused and rebuilds are faster. A `.dockerignore` file (Appendix D) keeps `node_modules`, `dist`, `.git` and local `.env` files out of the build context.

2.6.6 Exposing the Application Through a Mapped Port

Inside the container, Nginx listens on port 80. That port is not reachable from the host until it is published. The option `-p 8080:80` in `docker run` maps host port 8080 to container port 80, so the application opens at http://localhost:8080. The `EXPOSE 80` instruction in the Dockerfile only documents the port; it does not publish it. If port 8080 is already in use on the host, `docker run` fails with a "port is already allocated" error and a different host port must be chosen.

2.6.7 A Successful Build Is Not a Running Application

Each Docker stage proves something different. A successful `docker build` proves that the Dockerfile is valid and that `npm run build` completed, and the image then appears in `docker images`. It does not prove that the container starts: an invalid `nginx.conf`, for example, would build successfully but make the container exit immediately. A running container is shown by `docker ps` with a status of `Up`, and a working application is shown only by an HTTP response at http://localhost:8080 that renders the LMS interface. Section 3.2 verifies each step separately.

Table 2.6: Docker commands used in this project

Command	Purpose
`docker --version`	Confirm that the Docker CLI is installed
`docker build -t library-management-system:1.0 .`	Build the image from the Dockerfile in the current folder
`docker images`	List local images and confirm that the tag exists
`docker run -d --name library-management-docker -p 8080:80 library-management-system:1.0`	Start a detached container with port mapping
`docker ps`	List running containers, their status and their ports
`docker logs library-management-docker`	Inspect Nginx start-up and access logs

Command	Purpose
`docker stop library-management-docker` / `docker rm library-management-docker`	Stop and remove the container

2.7 Kubernetes Deployment and Orchestration

> **Implementation status: Configured but not yet executed** — the manifests exist in k8s/; evidence of a running deployment is recorded in Section 3.3.

2.7.1 Core Concepts

Kubernetes is an open-source system for running and managing containerised applications. Instead of starting containers by hand, the user declares the **desired state** (which image, how many copies, how they are reached) in YAML manifests. Kubernetes controllers then continuously compare the actual state of the cluster with the desired state and act to close any gap. This loop is called **reconciliation**. Table 2.7 summarises the objects used in this project.

Table 2.7: Kubernetes concepts used in this project

Concept	Meaning	Role in the LMS deployment
Cluster	A set of machines (nodes) managed by a Kubernetes control plane	The single-node cluster built into Docker Desktop (context `docker-desktop`)
Node	A machine that runs Pods	The Docker Desktop virtual machine
Pod	The smallest deployable unit: one or more containers sharing a network identity	Each Pod runs one Nginx container serving the LMS build
Deployment	Declares the Pod template and the desired number of replicas; manages rollouts	Keeps the requested number of LMS Pods running
ReplicaSet	Created by the Deployment to maintain the replica count	Replaces a Pod that is deleted or fails
Replica count	`spec.replicas`: how many identical Pods should exist	Demonstrates scaling and self-healing
Labels and selectors	Key–value tags and the queries that match them	`app: library-management` links the Service to the Pods
Service	A stable virtual IP and DNS name that load-balances across matching Pods	`library-management-service` on port 80

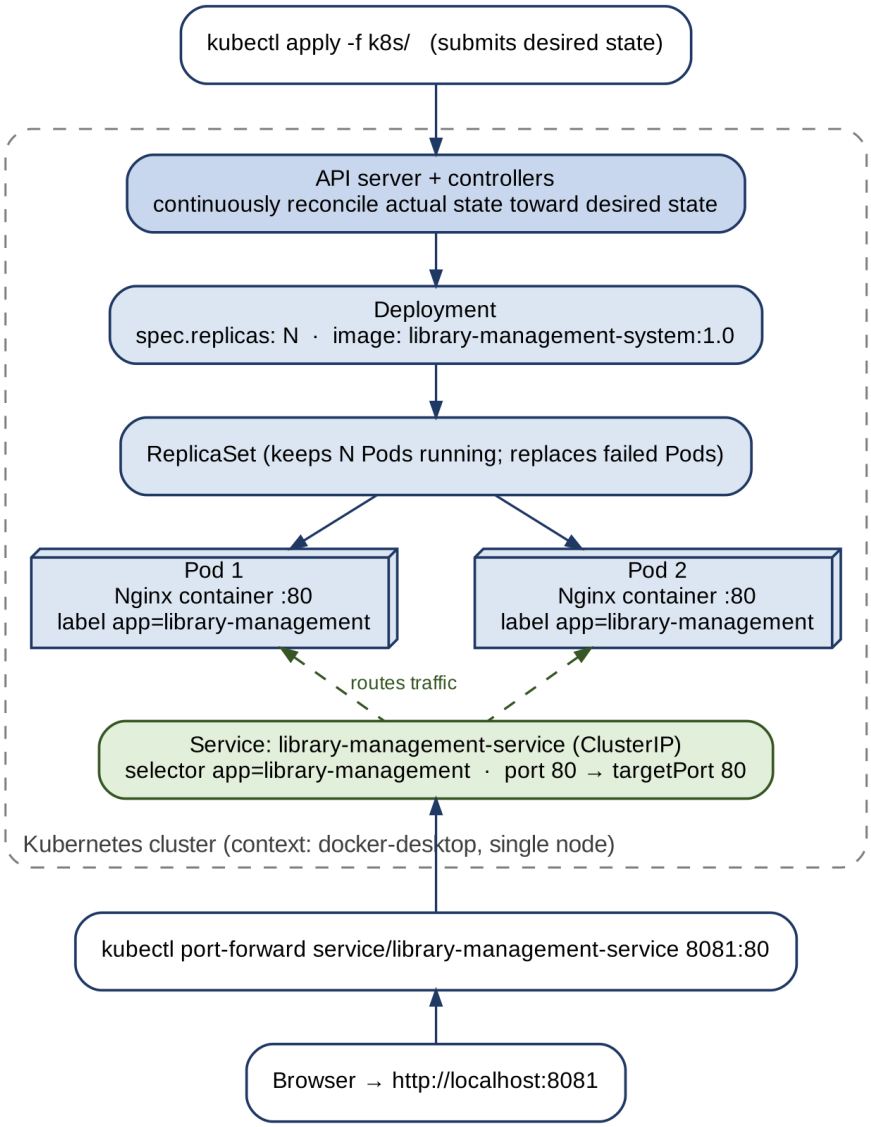


Figure 2.5: Kubernetes objects for the LMS and the path of a request from the browser to a Pod

2.7.2 Manifests

Listings 2.4 and 2.5 show representative manifests. *[Verify: check metadata names, labels, replica count, image tag and Service type against the files in k8s/]*

Listing 2.4: Representative Deployment manifest (k8s/deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: library-management-deployment
  labels:
    app: library-management
spec:
  replicas: 2
  selector:
    matchLabels:
      app: library-management
  template:
    metadata:
      labels:
        app: library-management
    spec:
      containers:
        - name: library-management
          image: library-management-system:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 10
```

Listing 2.5: Representative Service manifest (k8s/service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: library-management-service
spec:
  type: ClusterIP
  selector:
    app: library-management
  ports:
    - port: 80
      targetPort: 80
```

The Deployment's `selector.matchLabels` must match the labels in the Pod template, and the Service's `selector` must match the same labels; otherwise the Service has no endpoints and requests fail even though the Pods are running. The readiness probe asks Kubernetes to send an HTTP request to `/` and to mark a Pod as Ready only when Nginx answers successfully. If the repository manifest has no probe, Kubernetes treats a Pod as Ready as soon as its container is running.

2.7.3 Desired State, Replicas and Self-Healing

With `replicas: 2`, the Deployment's ReplicaSet ensures that two Pods with the label `app: library-management` exist at all times. If one Pod is deleted, for example with `kubectl delete pod <name>`, the controller observes that only one replica remains and creates a replacement automatically. Changing the image tag in the manifest and re-applying it triggers a rolling update, in which new Pods are started and old Pods are removed gradually. Because

the LMS Pods serve only static files and hold no data, all replicas are identical and any of them can serve any request.

2.7.4 Container Image Availability

A Pod can start only if the node can obtain the image. In this project the image is built locally and not pushed to a registry. With `imagePullPolicy: IfNotPresent`, Kubernetes uses an image that is already present on the node instead of pulling it. Whether a locally built image is visible to Docker Desktop's cluster depends on how the cluster was provisioned and which image store is used. If the Pods report `ErrImagePull` or `ImagePullBackOff`, the image is not available to the node. It must then be made available, for example by rebuilding it for the cluster's image store, loading it into the cluster, or pushing it to a registry and referencing the full registry path. `kubectl describe pod` shows the exact reason in its Events section.

2.7.5 Accessing the Application

A ClusterIP Service is reachable only from inside the cluster. For local demonstration, `kubectl port-forward service/library-management-service 8081:80` opens a tunnel from port 8081 on the developer's machine to port 80 of the Service, so the application opens at http://localhost:8081 for as long as the command keeps running. Alternatives are a `NodePort` Service, which opens a port in the 30000–32767 range on the node, or a `LoadBalancer` Service, which Docker Desktop publishes on localhost.

2.7.6 Configured, Running, Ready and Accessible

YAML files in a repository do not show that Kubernetes is working. Four distinct states must be established with real `kubectl` output:

Table 2.8: Distinguishing the stages of a Kubernetes deployment

State	What it means	How it is evidenced
Configured Deployment	Manifests exist and may have been accepted by the API server	Files in `k8s/`; `kubectl apply` prints "created" or "configured"
Running Pod	The container process has started on a node	`kubectl get pods` shows STATUS `Running`
Ready replica	The Pod passes its readiness check and receives Service traffic	READY `1/1` per Pod; Deployment READY `2/2`
Accessible application	A user can load the interface through the Service	The LMS UI renders at http://localhost:8081 during port-forward

2.8 Frontend Observability Using Grafana Faro

> **Implementation status: Configured but not yet executed** — Faro is initialised only when VITE_GRAFANA_FARO_URL is set at build time. Receipt of telemetry in Grafana must be confirmed (Section 3.4).

2.8.1 What Browser-Side Observability Means

Observability is the ability to understand what a system is doing from the signals it emits. For server applications these signals usually come from logs, metrics and traces produced on the server. In a frontend-only application, the code that matters runs on the user's device. If a JavaScript error breaks the issue form, or the dashboard loads slowly on a particular browser, nothing on the server records it, because Nginx only sees that the static files were downloaded. Browser-side observability, also called real user monitoring, closes this gap by collecting signals inside the browser and sending them to a backend for analysis.

2.8.2 The Faro Web SDK

Grafana Faro is an open-source JavaScript SDK from Grafana Labs for frontend observability. The core package for web applications is `@grafana/faro-web-sdk`; `@grafana/faro-web-tracing` adds distributed tracing and `@grafana/faro-react` adds React-specific integrations. When `initializeFaro()` is called at application start-up, the SDK registers a set of instrumentations, attaches metadata about the application, session, browser and page, batches the collected signals, and transports them over HTTPS to a collector URL. In Grafana Cloud, that collector feeds the **Frontend Observability** application, which presents errors, sessions, Web Vitals and events (Figure 2.6).

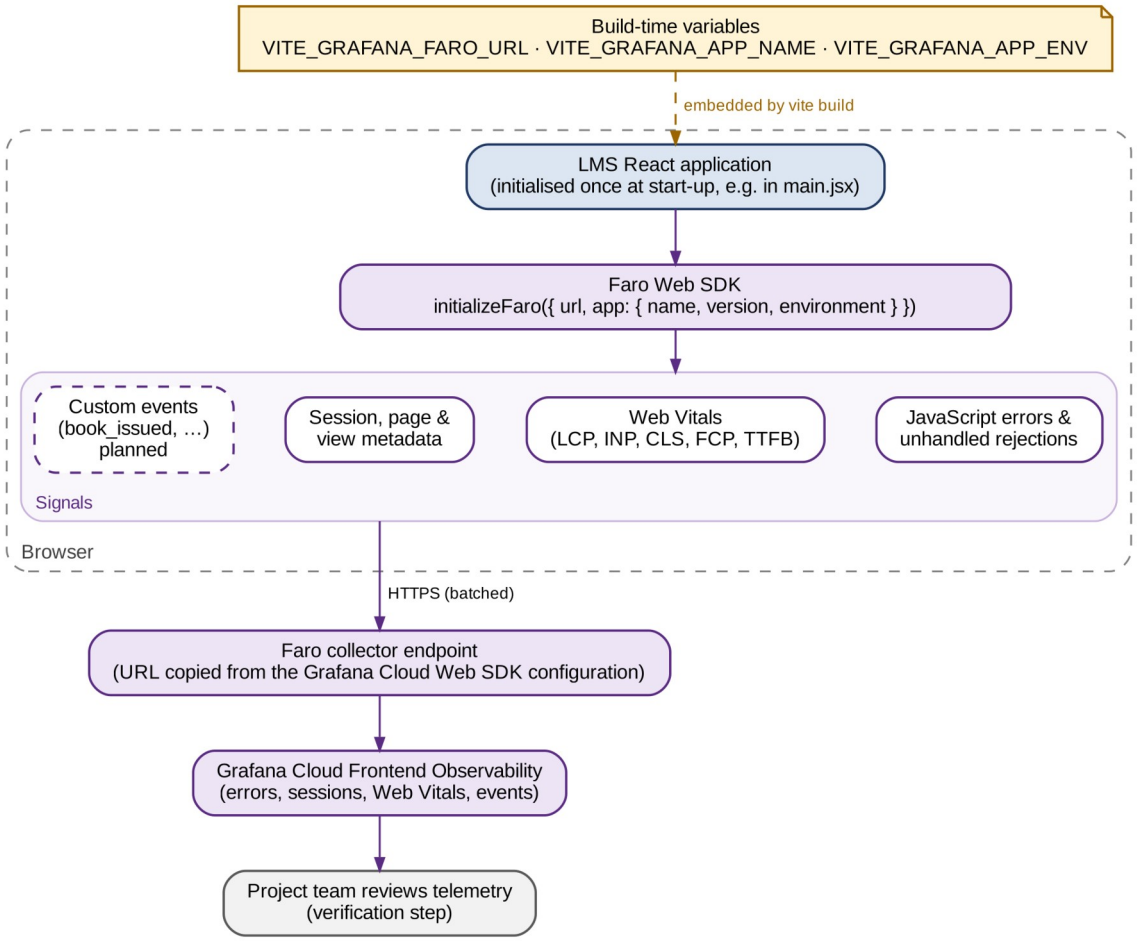


Figure 2.6: Grafana Faro telemetry flow from the browser to Grafana Cloud

Table 2.9: Signals collected by the Faro Web SDK

Signal	What is captured	Availability
JavaScript errors	Unhandled exceptions and unhandled promise rejections, with message, stack trace and page context	Default web instrumentation
Web Vitals	Largest Contentful Paint (LCP), Interaction to Next Paint (INP), Cumulative Layout Shift (CLS), First Contentful Paint (FCP), Time to First Byte (TTFB)	Enabled by default; browser support varies (CLS and INP are Chromium-only)
Sessions	A session identifier that groups all signals from one visit	Default
Page and view metadata	Current URL and view, used to attribute signals to pages	Default
Console messages	Console warn, info and error messages	Configurable
Custom events	Named application events sent with `faro.api.pushEvent()`	Requires explicit code; proposed for the LMS

2.8.3 Configuration Through Environment Variables

The collector URL and application identity are supplied through Vite environment variables rather than hard-coded. The real URL is copied from the **Web SDK configuration** page of the

application created in Grafana Cloud Frontend Observability. No placeholder URL can be used in its place, and the value below is intentionally left as a description.

Listing 2.6: Environment variables for Faro (.env.local, not committed to Git)

```
# Copy the collector URL from Grafana Cloud > Frontend Observability >
# <your app> > Web SDK configuration. Do not commit real values.
VITE_GRAFANA_FARO_URL=<collector URL from Grafana Cloud>
VITE_GRAFANA_APP_NAME=library-management-system
VITE_GRAFANA_APP_ENV=development
```

Listing 2.7: Representative Faro initialisation in src/main.jsx

```
import { initializeFaro, getWebInstrumentations } from '@grafana/faro-web-sdk';

const faroUrl = import.meta.env.VITE_GRAFANA_FARO_URL;

// Initialise only when a collector URL was provided at build time
if (faroUrl) {
  initializeFaro({
    url: faroUrl,
    app: {
      name: import.meta.env.VITE_GRAFANA_APP_NAME ?? 'library-management-system',
      version: '1.0.0',
      environment: import.meta.env.VITE_GRAFANA_APP_ENV ?? 'development',
    },
    instrumentations: [...getWebInstrumentations()],
  });
}
```

The guard around `initializeFaro()` makes the integration safe to leave in the code: when no URL is configured, the application runs normally and sends nothing. *[Verify: confirm where and how the repository initialises Faro]*

2.8.4 Security of VITE_ Variables

Vite exposes only variables prefixed with `VITE_` to client code through `import.meta.env`, and their values are written into the JavaScript bundle at build time. Anyone who opens the application can read them in the downloaded files. The Faro collector URL is designed to be used from browsers and is therefore expected to be visible, but private credentials such as API tokens, passwords or database connection strings must never be placed in `VITE_` variables. Real values belong in an untracked `.env.local` file or in build arguments, and the repository should contain only an `.env.example` with variable names.

2.8.5 Build-Time Configuration in Docker and Kubernetes

Because the values are embedded when `npm run build` runs, they must be present during the Docker build stage, for example through `ARG` instructions and `docker build --build-arg VITE_GRAFANA_FARO_URL=...`. Setting environment variables on the running Nginx container or in the Kubernetes Deployment has no effect, since the bundle has already been built. Changing the collector URL therefore requires rebuilding the image. This is a common source of confusion when Faro appears to send no data from a containerised build.

2.8.6 Custom Application Events (Proposed)

> **Implementation status: Proposed architecture or future enhancement** — claim these events only after the calls are present in the code and the events appear in Grafana.

Faro can record business-level events after successful operations, which would show how the application is actually used. Listing 2.8 shows the intended pattern and Table 2.10 lists the proposed events. Event attributes must describe the operation, never the person: member names, email addresses, phone numbers and identifiers must not be sent.

Listing 2.8: Proposed helper for custom events

```
import { faro } from '@grafana/faro-web-sdk';

export function trackEvent(name, attributes = {}) {
  // optional chaining: does nothing if Faro was not initialised
  faro.api?.pushEvent(name, attributes, 'library');
}

// Example: call only after issueBook() has completed successfully
trackEvent('book_issued', { module: 'circulation' });
```

Table 2.10: Proposed Faro custom events

Event name	Sent after	Permitted attributes (non-personal)
`book_added`	A book record is saved successfully	`module: books`
`book_deleted`	A book record is deleted	`module: books`
`member_added`	A member record is saved successfully	`module: members`
`member_deleted`	A member record is deleted	`module: members`
`book_issued`	An issue record is created	`module: circulation`
`book_returned`	An issue record is marked returned	`module: circulation`

2.8.7 Verifying That Grafana Receives Telemetry

1. Build the application with a real `VITE_GRAFANA_FARO_URL` and open it in a browser without privacy extensions that block analytics requests.
2. Open the browser's developer tools, select the Network tab and interact with the application. Confirm that requests are sent to the collector URL and return a successful (2xx) status.
3. Optionally trigger a harmless test error from the console, for example `setTimeout(() => { throw new Error('faro-test') })`, so that an unhandled error is reported.
4. In Grafana Cloud, open Frontend Observability, select the application and a time range that covers the test, and confirm that a session, the test error and Web Vitals values appear.
5. Capture the Grafana view as Figure 3.14. Only after this step may the integration be reported as "Implemented and verified".

2.9 Proposed Prometheus and Grafana Metrics Monitoring

> **Implementation status: Proposed architecture or future enhancement** — no metrics endpoint, Prometheus server, scrape configuration or Prometheus-backed dashboard exists in the current project.

This section explains Prometheus and Grafana in detail and designs how they could be added to the LMS. Three different things must be kept apart throughout:

Table 2.11: Three distinct monitoring situations

Situation	What it involves	Position in this project
Existing Grafana Faro frontend telemetry	Browser-side errors, Web Vitals, sessions sent to Grafana Cloud	Configured; verification pending (Section 3.4)
Proposed Prometheus metrics monitoring	A design for scraping service metrics and visualising them in Grafana	Described in this section; not implemented
A real, implemented and verified metrics system	A running metrics source, a Prometheus server with healthy targets, and dashboards showing collected data	Does not exist yet; required evidence is defined in Section 3.5

2.9.1 What Is Prometheus?

Prometheus is an open-source monitoring and alerting toolkit, originally developed at SoundCloud and now a graduated project of the Cloud Native Computing Foundation. It collects **numeric time-series metrics** from configured **targets** by periodically sending an HTTP request, conventionally to a `/metrics` endpoint, and parsing the plain-text response. This is a **pull** model: the monitored application does not push data to Prometheus; it exposes its current metric values and Prometheus fetches them on a schedule. The samples are stored in Prometheus's local time-series database and queried with the PromQL language.

Table 2.12: Prometheus terminology

Term	Explanation
Metric	A named numeric measurement, such as the total number of HTTP requests handled
Time series	The sequence of timestamped values for one metric name and one unique combination of labels
Metric name and labels	The name identifies what is measured; labels (key–value pairs such as `method="GET"`) distinguish dimensions. Every distinct label combination creates a separate time series
Sample	One value at one timestamp
Target	An endpoint that Prometheus scrapes, such as `library-service:8000/metrics`
Job	A named group of targets with the same purpose, configured in `scrape_configs`
Scrape interval	How often each target is scraped, for example every 15 seconds

Term	Explanation
Exporter	A separate program that translates metrics from a system that cannot expose Prometheus metrics itself into the Prometheus format
PromQL	Prometheus Query Language, used to select, aggregate and calculate over time series
Alerting rule	A PromQL condition evaluated by Prometheus; when it holds for a set duration, an alert is sent to Alertmanager for routing and notification

Table 2.13: Prometheus metric types

Type	Behaviour	Typical use
Counter	Only increases (resets to zero on restart)	Requests served, errors, books issued
Gauge	Can go up and down	Current number of issued books, memory in use
Histogram	Counts observations in configurable buckets, plus their sum and count	Request latency, used for percentiles
Summary	Calculates quantiles on the client side	Latency where server-side aggregation is not needed

A target exposes metrics in the text exposition format. The fragment below illustrates the format only; it is not output from this project.

Listing 2.9: Illustrative Prometheus exposition format (format example only)

```
# HELP http_requests_total Total HTTP requests handled.
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/api/books",http_status="200"} 3
```

Labels must have a small, bounded set of values. Using unbounded values such as member names, email addresses or record identifiers as labels would create one time series per value, which overloads Prometheus (the “high cardinality” problem) and, more importantly here, would expose personal data. Such data must never appear in metric labels.

Prometheus can only collect what some component exposes. An application must therefore either be **instrumented** with a Prometheus client library that publishes `/metrics`, or be accompanied by an **exporter** that converts its internal statistics into the Prometheus format. Without such a metrics source there is nothing to scrape.

2.9.2 What Is Grafana?

Grafana is an open-source visualisation and analytics platform. It does not store metrics itself; instead it connects to **data sources** such as Prometheus, Loki, databases or Grafana Cloud services and queries them on demand. Results are shown in **dashboards**, which are collections of **panels** such as time-series graphs, single-value stat panels, gauges and tables. Each panel has a query, written in the data source’s language (PromQL for Prometheus), and is evaluated over the dashboard’s **time range**, for example the last 15 minutes or the last 24 hours. Grafana

can also evaluate **alert rules** on query results and send notifications. Figure 2.7 shows how the two tools work together.

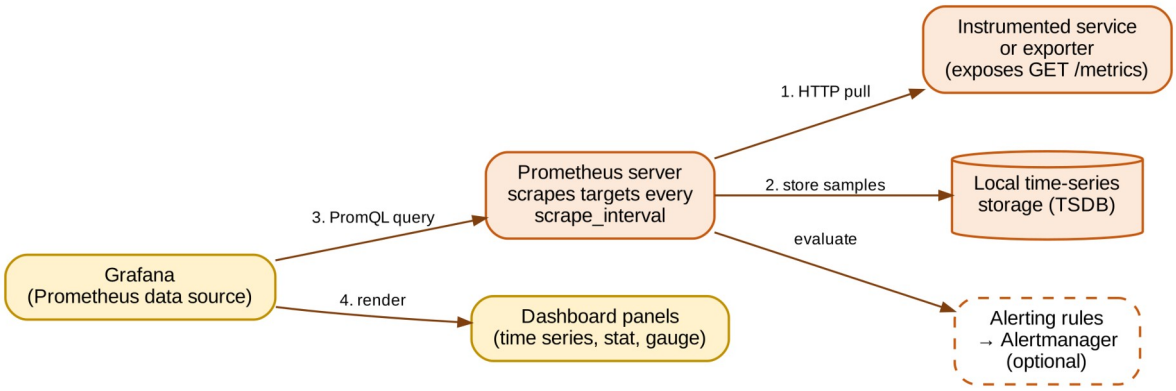


Figure 2.7: Generic Prometheus and Grafana data flow (concept)

1. An instrumented service or exporter exposes current metric values at `/metrics`.
2. Prometheus scrapes the target at each scrape interval.
3. Prometheus stores the samples as time series.
4. Grafana sends PromQL queries to Prometheus through its Prometheus data source.
5. Dashboard panels visualise the results over the selected time range.

2.9.3 Limitation of the Current Frontend-Only Architecture

The current LMS cannot be monitored by Prometheus in the conventional way, for four reasons:

- **There is no metrics endpoint.** The React application is a set of static files served by Nginx. No component of the application listens for metric scrapes or produces Prometheus-format output.
- **Browsers are not scrape targets.** The JavaScript code and the localStorage values exist inside each user's browser. Prometheus is a server that pulls from network endpoints; it cannot reach into browsers, call JavaScript functions or read localStorage, and browsers do not run an HTTP server that Prometheus could contact.
- **There is no single source of truth for library figures.** Every browser holds its own independent dataset. A metric such as "total books in the library" has no authoritative value to report until data is centralised.
- **Nginx does not expose Prometheus metrics by default.** Open-source Nginx can provide basic connection statistics through its `stub_status` module, and a separate exporter can convert these into Prometheus format. That would measure web-server activity, not library operations, and it is also not configured in this project.

Conventional Prometheus monitoring therefore requires a valid metrics source to be introduced first, such as a backend service instrumented with a Prometheus client library, an exporter that exposes appropriate metrics (for example an Nginx exporter, or Kubernetes components such

as kube-state-metrics for cluster-level metrics), or another explicitly designed collector that exposes supported metrics in a scrapeable format. None of these exists in the current repository.

2.9.4 Proposed Future Prometheus Architecture

Figure 2.8 shows the proposed extension. A future backend API or metrics exporter would sit between the frontend and the data, expose `/metrics`, and be registered as a Prometheus scrape target. Prometheus would store the scraped series, and Grafana dashboards would query them. Faro telemetry continues on its own path and complements the server-side metrics.

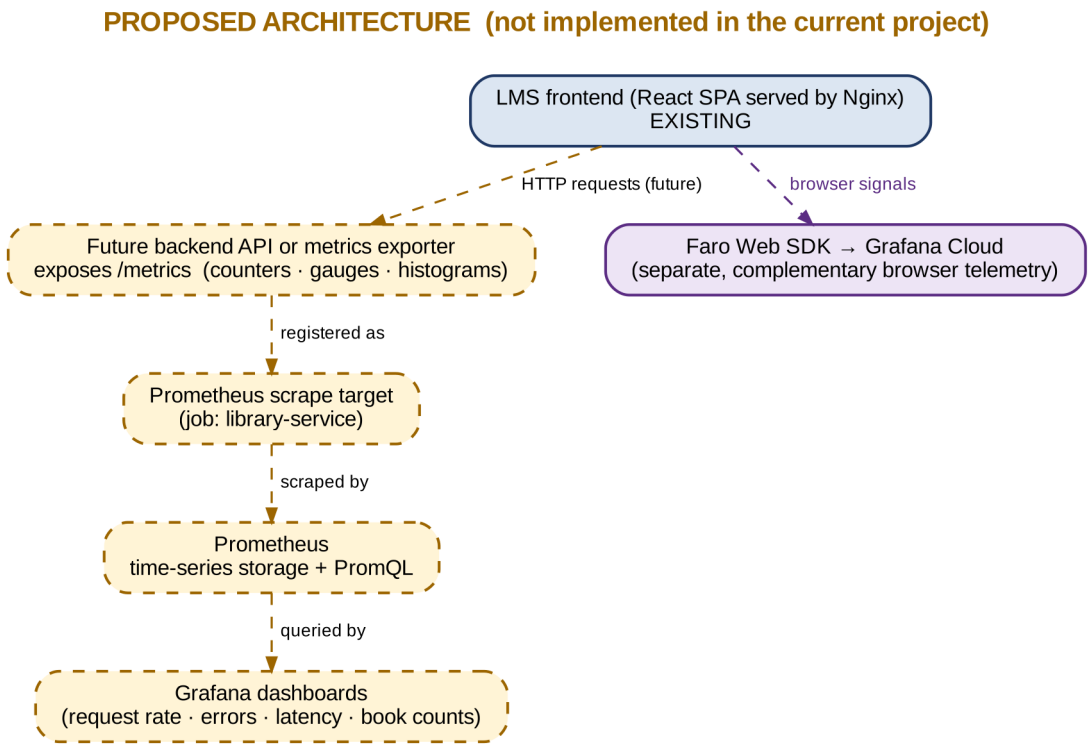


Figure 2.8: PROPOSED ARCHITECTURE: future Prometheus metrics flow for the LMS

Table 2.14 lists metrics such a backend might expose. These are **illustrative names**, not metrics that exist in the current application.

Table 2.14: Illustrative metrics for a future backend (not present in the current application)

Metric name	Type	Bounded labels	Meaning and required source
`http_requests_total`	Counter	`method`, `route`, `http_status`	Requests handled; needs HTTP middleware in a backend
`http_request_duration_seconds`	Histogram	`method`, `route`	Request latency; needs timing middleware in a backend
`http_errors_total`	Counter	`route`, `http_status`	Failed requests; can also be derived from `http_requests_total` filtered by status
`library_books_total`	Gauge	none	Books in the catalogue; needs a central database as the source of truth

Metric name	Type	Bounded labels	Meaning and required source
`library_books_issued_total`	Counter	none	Cumulative issue operations; a gauge such as `library_books_issued` would instead report books currently on loan

Request counters and latency histograms require a server or other component that actually handles requests. Book-count metrics require a defined source of truth, such as a central database, and a deliberate exporting design, for example computing the gauge from the database when Prometheus scrapes, or updating it whenever a record changes. Under the Prometheus naming conventions the `_total` suffix is reserved for counters, so a value that rises and falls with the number of books currently issued should be a gauge without that suffix.

2.9.5 Proposed Monitoring Architecture Using Docker and Kubernetes

Figure 2.9 shows how a future monitoring stack could be laid out on the same local Kubernetes cluster. It is a **deployment view**: it shows which workloads would exist and how they would reach each other, whereas Figure 2.8 shows the **telemetry flow**. The existing LMS frontend Deployment and Service would remain unchanged. A future API or exporter would run as its own Deployment with a Service on port 8000. Prometheus would run as a Deployment whose `prometheus.yml` is supplied through a ConfigMap, and Grafana would run as a Deployment with Prometheus configured as its data source. Both could be opened locally through `kubectl port-forward`. For quick experiments the same four components could also be started with Docker Compose.

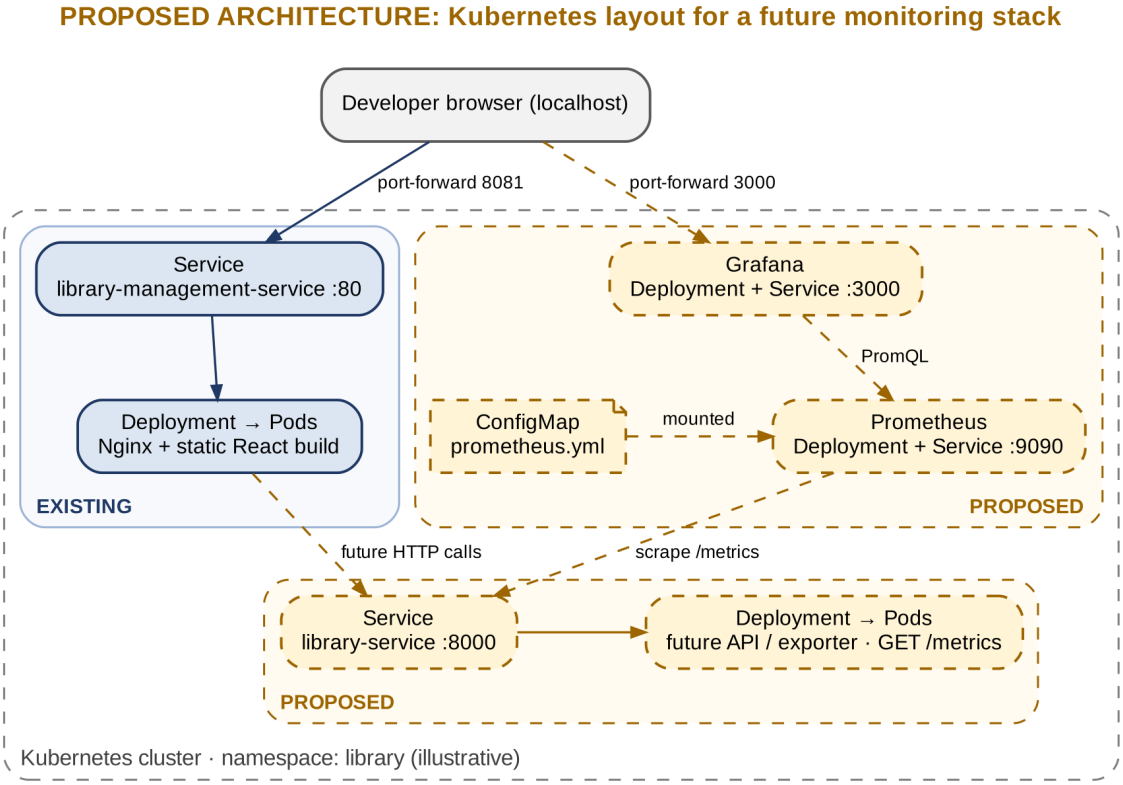


Figure 2.9: PROPOSED ARCHITECTURE: Kubernetes layout for a future monitoring stack

> **Note:** Drawing Prometheus and Grafana in a diagram does not mean the application is monitored. Monitoring exists only when a running target exposes metrics, Prometheus reports that target as UP, and queries return collected data.

2.9.6 Example Prometheus Configuration

Listing 2.10: EXAMPLE ONLY: prometheus.yml for the proposed design (not deployed)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"            # Prometheus monitoring itself
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "library-service"       # the FUTURE backend or exporter
    metrics_path: /metrics
    static_configs:
      - targets: ["library-service:8000"]
```

`scrape_interval` sets how often targets are scraped and `evaluation_interval` how often rules are evaluated. Each entry in `scrape_configs` defines a job and its targets. The target `library-service:8000` is valid only if a service with that DNS name exists, is reachable from the Prometheus Pod (for example in the same Kubernetes namespace), listens on port 8000 and returns Prometheus-format metrics at `/metrics`. In the current project no such

service exists. Prometheus would show the target as DOWN on its Status → Targets page, and the automatically generated metric `up{job="library-service"}` would be 0. This configuration has not been deployed.

2.9.7 Proposed Grafana Dashboard Panels

Table 2.15: Proposed dashboard panels and the instrumentation each requires

Panel	Visualisation	Query basis	Required data source and instrumentation
Request rate	Time series	`rate()` over `http_requests_total`	Prometheus; HTTP counter in a future backend
Error rate	Time series or stat	Share of 5xx responses	Prometheus; status label on the request counter
Request latency (p95)	Time series	`histogram_quantile()` over the duration histogram	Prometheus; latency histogram in the backend
Application availability	Stat	`up{job="library-service"}`	Prometheus; produced automatically for each scraped target
Total books	Stat	`library_books_total`	Prometheus; gauge exported from a central database
Issued books	Stat	`library_books_issued`	Prometheus; gauge exported from a central database
Container CPU and memory	Time series	Container resource metrics	Prometheus scraping kubelet/cAdvisor, with the required Kubernetes permissions

No populated dashboard is shown in this report because no metrics are being collected. Any screenshot of such a dashboard would have to come from a running implementation.

2.9.8 Example PromQL Queries

The following queries assume a future backend that exposes the illustrative metrics in Table 2.14 with an `http_status` label. Label names depend on the instrumentation library and code (other common choices are `status` or `code`), so the exact queries must be adjusted to the implemented labels.

Listing 2.11: Example PromQL for the proposed design (no data in the current project)

```
# Requests per second over the last 5 minutes, all routes
sum(rate(http_requests_total[5m]))

# Requests per second by route
sum by (route) (rate(http_requests_total[5m]))

# Fraction of requests that returned a 5xx status
sum(rate(http_requests_total{http_status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# 95th-percentile request latency from a histogram
histogram_quantile(0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

# Is the scrape target reachable? (1 = up, 0 = down)
up{job="library-service"}
```

`rate()` converts a constantly increasing counter into a per-second rate over the window in brackets, and `sum by (...)` aggregates the result across series. PromQL queries return useful results only when matching metrics are actually being collected. Against the current frontend-only project, every one of these queries would return an empty result, because the metrics do not exist.

2.9.9 Difference Between Faro and Prometheus

Table 2.16: Comparison of Grafana Faro and Prometheus

Aspect	Grafana Faro	Prometheus
Where data originates	Inside the user's browser	At server-side targets that expose metrics
Collection model	SDK pushes batched signals to a collector	Server pulls (scrapes) `/metrics` endpoints
Browser errors	Appropriate when instrumented	Not directly collected by itself
Browser performance	Appropriate when supported by Faro instrumentation (Web Vitals)	Requires a suitable metrics source or exporter
Server request metrics	Not a replacement for backend metrics instrumentation	Appropriate when an instrumented target exposes metrics
Scraping `/metrics`	Not its main role	Core collection model
Current project status	Configured; verification pending	Proposed; not implemented

Grafana can display data from many sources side by side, so Faro telemetry and Prometheus metrics are complementary rather than interchangeable. Faro answers "what did users experience in their browsers?", while Prometheus answers "how are the services behaving?". A mature version of the LMS would use both.

2.10 Git and GitHub Workflow

> **Implementation status: Implemented but not yet verified** — attach the repository and commit-history screenshot (Figure 3.5).

Git is a distributed version-control system. Every **commit** records a snapshot of the project together with an author, timestamp and message, so the history shows how the application evolved and allows any earlier state to be restored. **Branches** allow work, such as a new module, to proceed in isolation and be **merged** into the main branch when it is complete. GitHub hosts the **remote** repository: `git push` uploads local commits and `git pull` brings in others' changes, which lets the team collaborate and keeps an off-machine backup of the code.

The recommended workflow for the project keeps `main` in a working state, develops each feature on a short-lived branch (for example `feature/issue-return`), uses clear commit messages that describe the change, and merges through a pull request so that changes can be reviewed. A `.gitignore` file (Appendix D) excludes `node_modules/`, `dist/` and local `.env` files so that generated files and private configuration are never committed.

Table 2.17: Git commands used in the workflow

Command	Purpose
`git init` / `git clone <url>`	Create a repository or copy the remote repository
`git status`, `git add <files>`	Inspect and stage changes
`git commit -m "message"`	Record a snapshot with a descriptive message
`git checkout -b feature/<name>`	Create and switch to a feature branch
`git push -u origin <branch>`	Publish the branch to GitHub
`git pull`	Fetch and merge remote changes
`git log --oneline --graph`	View the commit history (used for Figure 3.5)

Specific commit counts, branch names and contributors are deliberately not stated here; they should be read from the actual repository and shown in Figure 3.5.

2.11 Jira and Scrum Workflow

> **Implementation status: Proposed architecture or future enhancement** — presented as a supporting project-management plan. If the team used Jira, replace this section with the real board, epics and issues and attach Figure 3.6.

Scrum is an agile framework in which work is organised into short, fixed-length iterations called sprints. The **product backlog** holds all planned work as user stories and tasks, grouped into **epics**. At the start of each sprint the team selects items for the **sprint backlog**, works on them, and moves each through the board columns To Do, In Progress and Done. A short review at the end of the sprint demonstrates completed work, and a retrospective identifies improvements. Jira supports this process with backlogs, boards, sprints and issue tracking.

Table 2.18: Proposed epic structure for the LMS backlog

Epic	Example stories and tasks
Library application	Book form and list; member form and list; issue and return; dashboard counts; localStorage service
Containerisation	Write the multi-stage Dockerfile; add nginx.conf with SPA fallback; build and run the container
Kubernetes deployment	Write Deployment and Service manifests; deploy to Docker Desktop; verify self-healing
Frontend observability	Create the Grafana Cloud application; initialise Faro; verify telemetry; add custom events
Metrics monitoring (future)	Introduce a metrics source; deploy Prometheus and Grafana; build dashboards
Documentation	Capture screenshots; complete test tables; prepare report and viva

No sprint totals, issue counts or velocity figures are reported, because they would have to come from an actual Jira project.

2.12 Continuous Integration with GitHub Actions or Jenkins

> **Implementation status: Proposed architecture or future enhancement** — no pipeline run is reported. If a pipeline was executed, add its real run screenshot as Figure 3.15.

Continuous integration (CI) automatically builds and checks the code whenever changes are pushed, so that a broken build is detected immediately rather than at demonstration time. GitHub Actions runs workflows defined in YAML files in `.github/workflows/`; Jenkins runs pipelines defined in a `Jenkinsfile` on a Jenkins server. Four stages must be distinguished when reporting CI:

Table 2.19: Stages of CI evidence

Stage	Meaning	Evidence
Pipeline configuration file	A workflow or Jenkinsfile exists in the repository	The file itself
Executed pipeline	The CI system has actually run the pipeline	A run entry in the Actions tab or Jenkins build history
Successful build	All steps of that run completed without error	Green status for the run, with logs
Successful deployment	The built artefact was released to a target environment	Deployment job logs and a reachable application

Listing 2.12: Proposed GitHub Actions workflow (.github/workflows/ci.yml, not yet executed)

```
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - run: docker build -t library-management-system:ci .
```

This workflow would verify that dependencies install, that the production build succeeds and that the Docker image builds on every push and pull request. It does not deploy anything, because the project has no shared deployment environment. Test and lint steps should be added only once the corresponding npm scripts exist.

Chapter 3
Results and Discussion

This chapter records the results of the project. In keeping with the status classification in Section 1.4, a result is reported only when it is supported by observed evidence: a screenshot, terminal output or Grafana view. For every test, the objective, procedure and expected result are defined in advance. The **Actual result** and **Status** columns are intentionally left blank and are to be completed by the team while executing each test, using Pass, Fail or Not executed. The screenshots referred to in this chapter are collected in the reserved space at the end of the chapter.

3.1 Application Functional Results

Table 3.1: Functional test cases for the LMS application

ID	Test objective	Procedure	Expected result	Actual result	Status
TC-01	Dashboard loads	Open the application URL	Dashboard shows summary counts consistent with stored records (Figure 3.1)		
TC-02	Add a book	Fill the book form with valid data and save	Book appears in the list; total and available counts increase by one (Figure 3.2)		
TC-03	Required-field validation	Submit the book form with the title empty	Record is not saved; a message is shown (if validation is implemented)		
TC-04	Delete a book	Delete an available book	Book is removed; dashboard totals decrease		
TC-05	Add a member	Fill the member form and save	Member appears in the list; member count increases (Figure 3.3)		
TC-06	Delete a member	Delete a member with no active issue	Member is removed; member count decreases		
TC-07	Issue an available book	Select an available book and a member; issue	Issue record created; book unavailable; issued count increases (Figure 3.4)		
TC-08	Block issue of unavailable book	Try to issue the book issued in TC-07	Issue is refused with a message; no duplicate record		
TC-09	Return a book	Return the issue record from TC-07	Record marked returned; book available; issued count decreases		
TC-10	Persistence after refresh	Add data, then reload the page	All records are still present		
TC-11	Persistence scope	Open the same URL in a private window or another browser	Empty dataset (expected: storage is per origin and profile)		

ID	Test objective	Procedure	Expected result	Actual result	Status
TC-12	Storage inspection	Developer tools > Application > Local Storage	Keys contain JSON arrays of books, members and issue records		

Tests TC-10 and TC-11 are particularly important for the viva because together they demonstrate both the strength (persistence across refreshes) and the main limitation (no sharing across browsers) of the localStorage design.

3.2 Docker Verification

The Docker results are recorded step by step so that building an image, starting a container and serving the application are each evidenced separately (Section 2.6.7). Commands assume that the container listens on port 80 and is published on host port 8080, as in Listing 2.2. *[Verify: confirm ports against the Dockerfile and nginx.conf]*

Table 3.2: Docker verification

Operation	Command	Expected observation	Actual result	Verification status
Check installation	`docker --version`	Docker version string is printed		
Build image	`docker build -t library-management-system:1.0 .`	Build stage runs `npm run build`; output ends with the image being named and tagged (Figure 3.7)		
List image	`docker images`	`library-management-system` with tag `1.0` is listed		
Run container	`docker run -d --name library-management-docker -p 8080:80 library-management-system:1.0`	A container ID is printed		
Check container	`docker ps`	Container shows STATUS `Up` and PORTS `0.0.0.0:8080->80/tcp` (Figure 3.8)		
Open application	Browser: http://localhost:8080	LMS dashboard renders (Figure 3.9)		
Check logs	`docker logs library-management-docker`	Nginx start-up messages and `GET /` requests with status 200		
SPA fallback	Reload the browser on a client-side route (if routing is used)	The application is shown instead of an Nginx 404 page		

3.3 Kubernetes Verification

Kubernetes results must come from real `kubectl` output. The table separates the configured, running, ready and accessible states defined in Table 2.8. *[Verify: confirm resource names and ports against the files in k8s/]*

Table 3.3: Kubernetes verification

ID	Objective	Command	Expected result	Actual result	Status
KB-01	Correct cluster	`kubectl config current-context`	`docker-desktop`		
KB-02	Node ready	`kubectl get nodes`	Node listed with STATUS `Ready` (Figure 3.10)		
KB-03	Apply manifests	`kubectl apply -f ./k8s/`	Deployment and Service reported as created or configured		
KB-04	Deployment ready	`kubectl get deployments`	READY equals the replica count, e.g. `2/2` (Figure 3.11)		
KB-05	Pods running and ready	`kubectl get pods`	Each Pod STATUS `Running`, READY `1/1`		
KB-06	Service exists	`kubectl get services`	`library-management-service` with port `80/TCP` (Figure 3.12)		
KB-07	Overview	`kubectl get all`	Deployment, ReplicaSet, Pods and Service listed together		
KB-08	Access application	`kubectl port-forward service/library-management-service 8081:80`	“Forwarding from 127.0.0.1:8081 -> 80”; UI loads at http://localhost:8081 (Figure 3.13)		
KB-09	Self-healing	`kubectl delete pod <pod-name>`, then `kubectl get pods`	A replacement Pod is created; Deployment returns to full READY count		
KB-10	Diagnose failures	`kubectl describe pod <pod-name>`	Events explain any problem, e.g. `ImagePullBackOff`		

3.4 Frontend Observability Results

Faro is reported as working only when telemetry is seen in Grafana Cloud. The procedure follows Section 2.8.7.

Table 3.4: Grafana Faro verification

ID	Test objective	Procedure	Expected result	Actual result	Status
FO-01	Configuration present	Confirm the Faro URL variable was set for the build (do not print its value)	Collector URL embedded at build time		
FO-02	SDK sends data	Developer tools > Network while using the app	Requests to the collector URL with 2xx status		
FO-03	Session visible	Grafana Cloud > Frontend Observability > application	A session appears for the test time range (Figure 3.14)		

ID	Test objective	Procedure	Expected result	Actual result	Status
FO-04	Error captured	Trigger the test error from Section 2.8.7	Error `faro-test` listed with stack trace		
FO-05	Web Vitals	Load pages and interact; check the Web Vitals view	Values for supported metrics (e.g. LCP, FCP, TTFB) appear		
FO-06	Custom events	Issue a book after events are implemented	Event `book_issued` visible (not applicable until implemented)		

3.5 Proposed Prometheus and Grafana Monitoring: Status and Design

Implementation status: Proposed architecture or future enhancement

Prometheus-based monitoring has not been implemented, so this section reports its status and design rather than results. No successful scrapes, metric values, populated Prometheus panels, fired alerts or performance improvements are reported, because none have been observed.

Table 3.5: Prometheus and Grafana implementation status

Item	Current status	Required before it can be verified
Metrics endpoint (`/metrics`)	Not present	A backend instrumented with a Prometheus client library, or a suitable exporter
Prometheus server	Not deployed	Prometheus Deployment or container with a configuration file
Scrape configuration	Example only (Listing 2.10)	Configuration pointing to a real, reachable target
Targets page	Not applicable	Target shown as UP on Status > Targets
PromQL results	Not applicable	Queries such as `up{job="library-service"}` returning data
Grafana Prometheus data source	Not configured	Data source added; “Save & test” succeeds
Dashboards	Designed (Table 2.15); not populated	Panels showing collected data
Alert rules	Not defined	Rules written and tested against real metrics

When the extension is implemented, it should be verified in this order: (1) the metrics endpoint returns Prometheus-format text when opened directly; (2) the Prometheus Status → Targets page shows the target as UP; (3) `up{job="library-service"}` returns 1; (4) the example queries in Listing 2.11 return series; (5) Grafana’s Prometheus data source test succeeds; and (6) dashboard panels display data. Screenshots of the Targets page, a query result and a populated dashboard should be added at that point, but not before.

3.6 Discussion

The application brings the catalogue, member register and circulation log into a single interface. Because the dashboard figures are derived from the same records that the other modules modify, the inconsistency between separate registers described in Section 1.2 cannot arise within one dataset. Isolating every storage operation in the service layer keeps business rules, such as blocking the issue of an unavailable book, in one place, and it leaves a clean seam for replacing localStorage with a backend later.

The frontend-only design has clear benefits for a mini project: it needs no server, database or hosting cost, it can be demonstrated offline, and it keeps attention on the workflows. Its costs are equally clear. Data belongs to one browser profile, cannot be shared between librarians, and is not protected by any access control. The origin-scoped behaviour of localStorage also explains why the Vite development server, the Docker container and the Kubernetes port-forward each show different data, which is a useful practical lesson in how web storage works.

The delivery pipeline shows how the same application can be packaged and operated reproducibly. The multi-stage Dockerfile produces a small runtime image that runs identically on any machine with Docker. Kubernetes adds declarative management: the desired replica count is restored automatically when a Pod is deleted, and a Service provides a stable endpoint regardless of which Pod answers. It is equally important to state what these steps do not provide. Running several replicas improves the availability of the web server but does nothing for data, which still resides in each user's browser.

The observability work shows that monitoring tools must match where the code runs. In this project the business logic runs in the browser, so browser-side telemetry through Faro is the appropriate source of information about errors and user experience. Prometheus, by contrast, needs a server-side target to scrape. Studying it in detail clarified the architectural change a production system would require (a backend with a central data store and an instrumented `/metrics` endpoint) and showed why simply adding Prometheus and Grafana containers would not, on its own, monitor anything.

No performance, reliability or scalability figures are claimed in this report. Such claims would require reproducible measurements, which are outside the scope of the current evidence.

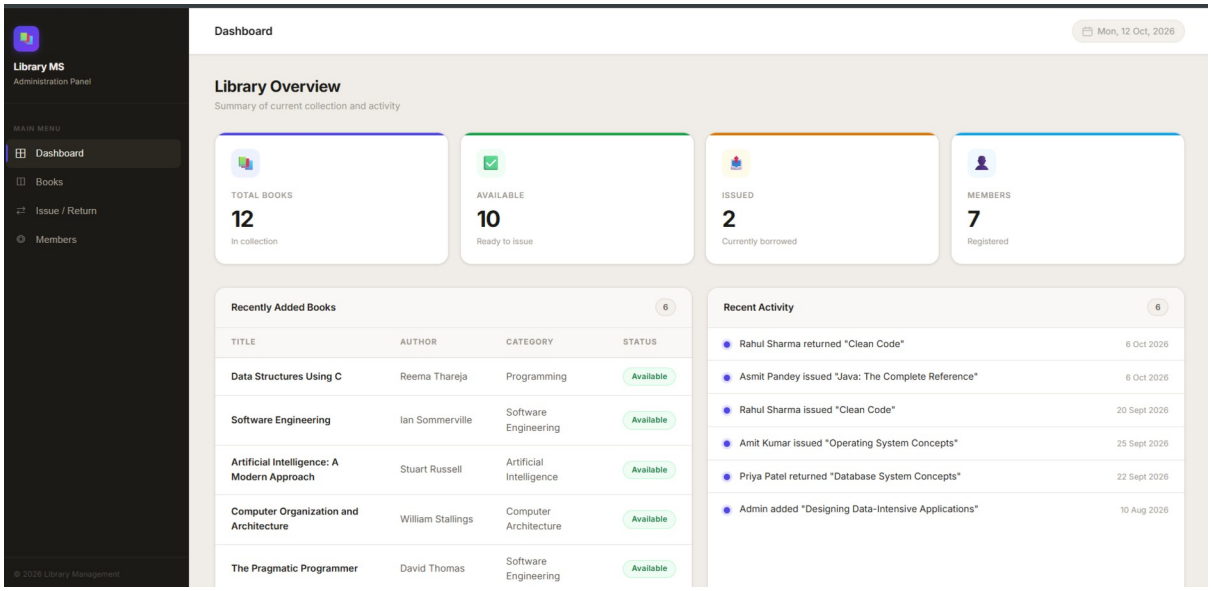
3.7 Limitations and Technical Considerations

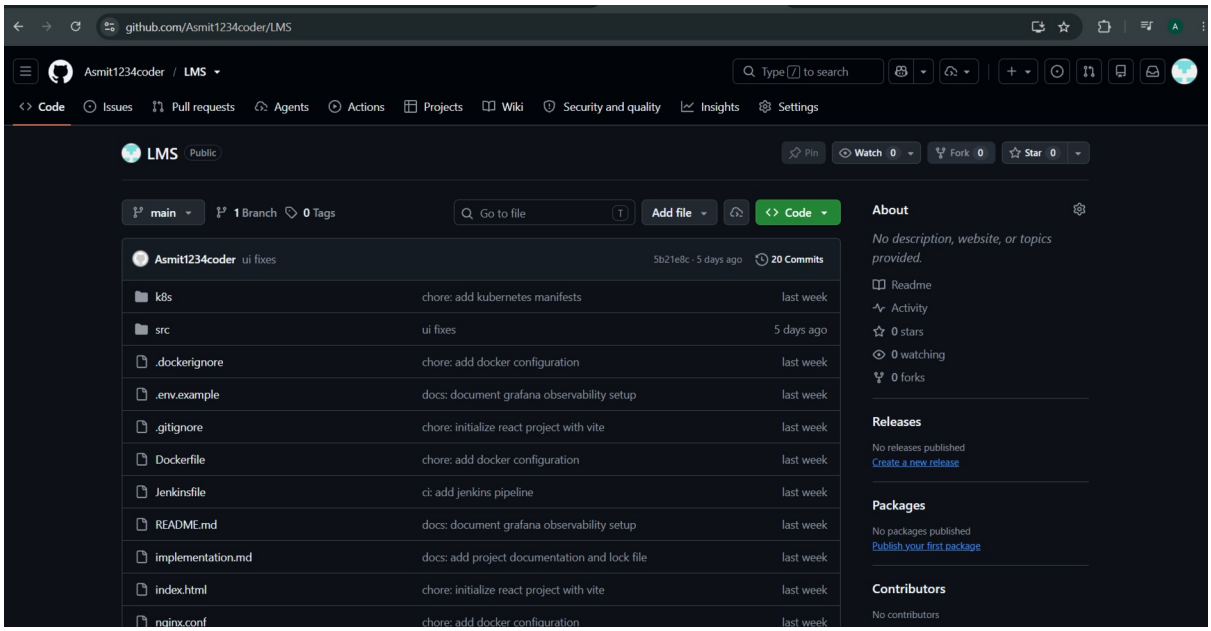
- **Durability.** Data can be lost by clearing browser data and has no backup. An export and import feature would be a simple mitigation.
- **Security and privacy.** Member details are stored unencrypted in the browser and are editable through developer tools. Personal data must also be kept out of telemetry and metrics.
- **Concurrency.** Two tabs editing the same data can overwrite each other's changes (the last write wins).
- **Scale.** The storage quota and synchronous access make localStorage unsuitable for large catalogues.

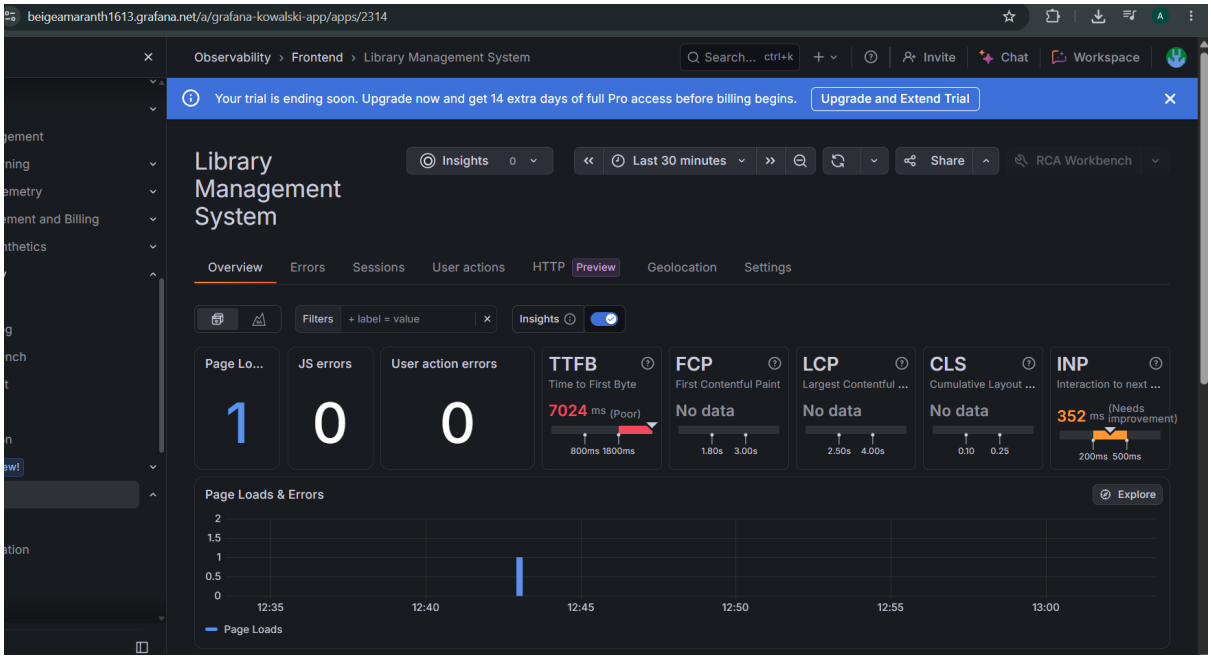
- **Image distribution.** The image exists only locally; a registry and versioned tags would be needed for any shared or cloud deployment.
- **Build-time configuration.** `VITE_` values are fixed when the image is built, so environment-specific configuration requires separate builds.
- **Telemetry reliability.** Ad blockers and network restrictions can prevent Faro data from arriving, so missing data does not always mean that the application is unused.
- **Testing.** Verification is currently manual; automated unit and end-to-end tests are not yet part of the project.
- **Metrics monitoring.** Prometheus and Grafana metrics monitoring remains a proposed extension until a scrapeable metrics source exists.

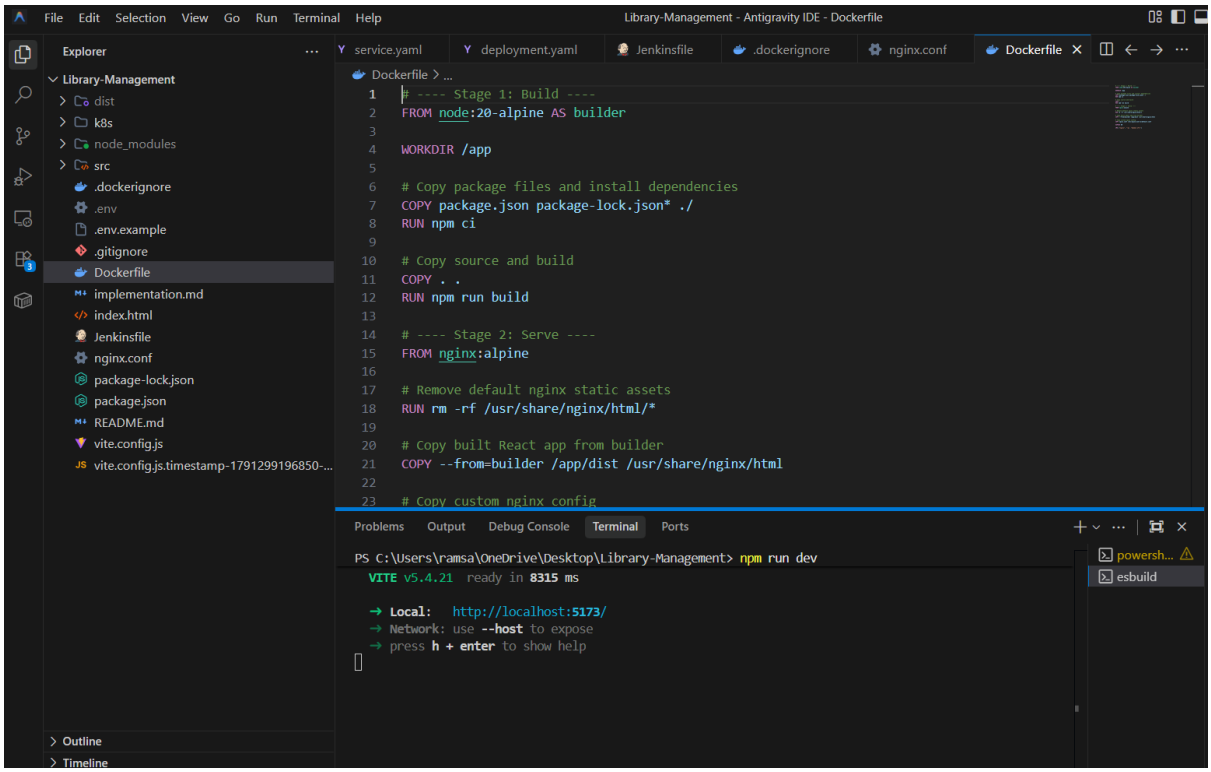
SCREENSHOTS — RESERVED SPACE

Each frame below is reserved for a real screenshot captured while executing the tests in this chapter. To insert one, click inside the frame, delete the guidance text and insert the image. Frames marked “include only if used” should be deleted if the corresponding tool was not used. No operational screenshot is provided for the proposed Prometheus extension; its design is shown in Figures 2.8 and 2.9 instead.


Library MS
Administration Panel
MAIN MENU
Dashboard
Books
Issue / Return
Members
© 2026 Library Management
Dashboard
Mon, 12 Oct, 2026
Library Overview
Summary of current collection and activity
TOTAL BOOKS
12
In collection
AVAILABLE
10
Ready to issue
ISSUED
2
Currently borrowed
MEMBERS
7
Registered
Recently Added Books
6
TITLE
AUTHOR
CATEGORY
STATUS
Data Structures Using C
Reema Thareja
Programming
Available
Software Engineering
Ian Sommerville
Software Engineering
Available
Artificial Intelligence: A Modern Approach
Stuart Russell
Artificial Intelligence
Available
Computer Organization and Architecture
William Stallings
Computer Architecture
Available
The Pragmatic Programmer
David Thomas
Software Engineering
Available
Recent Activity
6
Rahul Sharma returned "Clean Code"
6 Oct 2026
Asmit Pandey issued "Java: The Complete Reference"
6 Oct 2026
Rahul Sharma issued "Clean Code"
20 Sept 2026
Amit Kumar issued "Operating System Concepts"
25 Sept 2026
Priya Patel returned "Database System Concepts"
22 Sept 2026
Admin added "Designing Data-Intensive Applications"
10 Aug 2026


github.com/Asmit1234coder/LMS
Asmit1234coder / LMS
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
LMS
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
Asmit1234coder ui fixes
5b21e8c · 5 days ago
20 Commits
k8s
chore: add kubernetes manifests
last week
src
ui fixes
5 days ago
.dockerignore
chore: add docker configuration
last week
.env.example
docs: document grafana observability setup
last week
.gitignore
chore: initialize react project with vite
last week
Dockerfile
chore: add docker configuration
last week
Jenkinsfile
ci: add jenkins pipeline
last week
README.md
docs: document grafana observability setup
last week
implementation.md
docs: add project documentation and lock file
last week
index.html
chore: initialize react project with vite
last week
nginx.conf
chore: add docker configuration
last week
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors
No contributors


beigeamaranth1613.grafana.net/a/grafana-kowalski-app/apps/2314
Observability › Frontend › Library Management System
Search... ctrl+k
Invite
Chat
Workspace
Your trial is ending soon. Upgrade now and get 14 extra days of full Pro access before billing begins.
Upgrade and Extend Trial
Library Management System
Insights 0
Last 30 minutes
Share
RCA Workbench
Overview
Errors
Sessions
User actions
HTTP Preview
Geolocation
Settings
Filters + label = value
Insights
Page Lo...
1
JS errors
0
User action errors
0
TTFB
Time to First Byte
7024 ms (Poor)
800ms 1800ms
FCP
First Contentful Paint
No data
1.80s 3.00s
LCP
Largest Contentful ...
No data
2.50s 4.00s
CLS
Cumulative Layout ...
No data
0.10 0.25
INP
Interaction to next ...
352 ms (Needs improvement)
200ms 500ms
Page Loads & Errors
Explore
2
1.5
1
0.5
0
12:35
12:40
12:45
12:50
12:55
13:00
Page Loads


File Edit Selection View Go Run Terminal Help
Library-Management - Antigravity IDE - Dockerfile
Explorer
Library-Management
dist
k8s
node_modules
src
.dockerignore
.env
.env.example
.gitignore
Dockerfile
implementation.md
index.html
Jenkinsfile
nginx.conf
package-lock.json
package.json
README.md
vite.config.js
vite.config.js.timestamp-1791299196850-...
Outline
Timeline
service.yaml
deployment.yaml
Jenkinsfile
.dockerignore
nginx.conf
Dockerfile
Dockerfile > ...
1 # ---- Stage 1: Build ----
2 FROM node:20-alpine AS builder
3
4 WORKDIR /app
5
6 # Copy package files and install dependencies
7 COPY package.json package-lock.json* ./
8 RUN npm ci
9
10 # Copy source and build
11 COPY . .
12 RUN npm run build
13
14 # ---- Stage 2: Serve ----
15 FROM nginx:alpine
16
17 # Remove default nginx static assets
18 RUN rm -rf /usr/share/nginx/html/*
19
20 # Copy built React app from builder
21 COPY --from=builder /app/dist /usr/share/nginx/html
22
23 # Copy custom nginx config
Problems Output Debug Console Terminal Ports
PS C:\Users\ramsa\OneDrive\Desktop\Library-Management> npm run dev
VITE v5.4.21 ready in 8315 ms
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
powersh...
esbuild

Chapter 4
Conclusion and Future Scope

4.1 Conclusion

The Library Management System brings the core activities of a small library (maintaining books and members, issuing and returning books, and reviewing the current status) into one browser-based application built with React and Vite. A frontend service layer organises all data operations and persists records in localStorage, which keeps the application simple and self-contained while making its limits explicit: data belongs to a single browser profile and is neither shared nor secured by a server.

Around the application, the project demonstrates a practical DevOps workflow: version control with Git and GitHub, packaging of the production build into a multi-stage Docker image served by Nginx, declarative deployment to a local Kubernetes cluster through a Deployment and a Service, and browser-side observability with Grafana Faro where configured. The report also provides a detailed study of Prometheus and Grafana. It explains that conventional Prometheus monitoring is a future enhancement, because the current architecture has no scrapeable metrics source, and it specifies the backend instrumentation, scrape configuration, PromQL queries and dashboards that would be needed.

Throughout, each capability has been labelled with its implementation status and paired with a verification procedure. The project therefore demonstrates frontend development, browser-side persistence, version control, containerisation, orchestration and frontend observability, while keeping a clear distinction between what has been built and what has been designed.

Table 4.1: Completed work and future work

Completed in this project (subject to the verification in Chapter 3)	Future work (not implemented)
React and Vite application with dashboard, books, members and issue/return	Backend API and central database
localStorage persistence through a frontend service layer	Authentication and role-based authorisation
Git history and GitHub remote repository	Automated unit and end-to-end tests
Multi-stage Dockerfile with Nginx runtime	CI/CD pipeline and container registry
Kubernetes Deployment and Service manifests for Docker Desktop	Backend metrics instrumentation, Prometheus and Grafana dashboards
Grafana Faro Web SDK integration (configured)	Faro custom events, alerts and service-level objectives

4.2 Future Enhancements

1. **Backend API.** Introduce a REST API (for example with Node.js and Express, or Python and FastAPI) and replace the bodies of the existing service functions with HTTP calls, keeping their signatures unchanged.
2. **Centralised database.** Store books, members and issue records in a relational database such as PostgreSQL so that all librarians share one source of truth, with constraints that preserve referential integrity.
3. **Authentication and authorisation.** Add librarian and administrator accounts, role-based permissions and server-side validation.
4. **Automated tests.** Add unit tests for the service layer (for example with Vitest), component tests with React Testing Library, and end-to-end tests of the issue and return workflow (for example with Playwright).
5. **CI/CD.** Run the workflow in Listing 2.12 on every push and extend it to publish images and deploy automatically.
6. **Container registry.** Push versioned images to a registry such as Docker Hub or GitHub Container Registry, and reference immutable tags in the Kubernetes manifests.
7. **Backend metrics instrumentation.** Expose `/metrics` from the backend with request counters, latency histograms and library gauges with bounded labels (Section 2.9.4).
8. **Prometheus and Grafana dashboards.** Deploy the stack shown in Figure 2.9 and build the panels in Table 2.15 from real data.
9. **Alerts and service-level objectives.** Define objectives, for example for availability and request latency, and alert rules that notify the team when they are at risk. Targets should be set from measured baselines.
10. **Enhanced frontend observability.** Add the custom events in Table 2.10, distributed tracing across frontend and backend with `@grafana/faro-web-tracing`, and dashboards that combine Faro and Prometheus data.
11. **Functional extensions.** Search and filtering, due dates and overdue reminders, fines, data export and import, and an Ingress for cleaner access to the cluster.

Chapter 5

References

1. React Documentation — https://react.dev/
2. Vite Guide — https://vite.dev/guide/
3. Vite: Env Variables and Modes — https://vite.dev/guide/env-and-mode
4. MDN Web Docs: Window.localStorage — https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
5. MDN Web Docs: Web Storage API — https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
6. Git Documentation — https://git-scm.com/doc
7. GitHub Docs — https://docs.github.com/
8. Docker Documentation — https://docs.docker.com/
9. Docker: Multi-stage builds — https://docs.docker.com/build/building/multi-stage/
10. Docker Desktop: Kubernetes — https://docs.docker.com/desktop/features/kubernetes/
11. Kubernetes Documentation — https://kubernetes.io/docs/home/
12. Kubernetes: Deployments — https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
13. Kubernetes: Service — https://kubernetes.io/docs/concepts/services-networking/service/
14. NGINX Documentation — https://nginx.org/en/docs/
15. Grafana Cloud Frontend Observability — https://grafana.com/docs/grafana-cloud/observe-and-act/monitor-applications/frontend-observability/
16. Grafana Faro Web SDK (GitHub) — https://github.com/grafana/faro-web-sdk
17. Grafana Frontend Observability: Send custom events — https://grafana.com/docs/grafana-cloud/observe-and-act/monitor-applications/frontend-observability/instrument/custom-signals/events/
18. web.dev: Web Vitals — https://web.dev/articles/vitals
19. Prometheus: Overview — https://prometheus.io/docs/introduction/overview/
20. Prometheus: Metric types — https://prometheus.io/docs/concepts/metric_types/
21. Prometheus: Metric and label naming — https://prometheus.io/docs/practices/naming/
22. Prometheus: Querying basics (PromQL) — https://prometheus.io/docs/prometheus/latest/querying/basics/
23. Prometheus: Configuration — https://prometheus.io/docs/prometheus/latest/configuration/configuration/
24. NGINX Prometheus Exporter — https://github.com/nginx/nginx-prometheus-exporter
25. Grafana Documentation — https://grafana.com/docs/grafana/latest/
26. Grafana: Prometheus data source — https://grafana.com/docs/grafana/latest/datasources/prometheus/

27. Jira Cloud Support Documentation — https://support.atlassian.com/jira-software-cloud/
28. GitHub Actions Documentation — https://docs.github.com/en/actions
29. Jenkins User Documentation — https://www.jenkins.io/doc/

Appendices

Appendix A: Screenshot Checklist

Table A.1: Screenshot checklist

No.	Screenshot	Where to capture	Inclusion rule	Figure
1	Library dashboard	Running application	Required	3.1
2	Book management	Running application	Required	3.2
3	Member management	Running application	Required	3.3
4	Issue/return workflow	Running application	Required	3.4
5	GitHub repository and commit history	GitHub / terminal	Required	3.5
6	Jira board	Jira	Only if Jira was used	3.6
7	Successful Docker build	Terminal	Required	3.7
8	Docker Desktop running container	Docker Desktop	Required	3.8
9	LMS at localhost:8080	Browser	Required	3.9
10	Kubernetes node status	Terminal	Required	3.10
11	Kubernetes Deployment and Pods	Terminal	Required	3.11
12	Kubernetes Service	Terminal	Required	3.12
13	LMS opened through Kubernetes	Terminal and browser	Required	3.13
14	Grafana Frontend Observability with real telemetry	Grafana Cloud	Only once telemetry is received	3.14
15	Successful Jenkins or GitHub Actions run	CI system	Only if a pipeline was run	3.15
16	Prometheus targets page	Prometheus UI	Only if Prometheus is actually configured	Add when implemented
17	Prometheus query result	Prometheus UI	Only if a metrics source is available	Add when implemented
18	Grafana Prometheus dashboard	Grafana	Only if populated with real data	Add when implemented

Appendix B: Additional Test Cases

These edge cases complement the functional tests in Table 3.1. Their purpose is to record how the application actually behaves; the expected behaviour describes a robust design and may reveal improvements.

Table B.1: Edge-case test cases

ID	Scenario	Procedure	Robust expected behaviour	Actual result	Status
EC-01	Corrupted storage	Developer tools: set a key to invalid JSON, then reload	Application loads with an empty list instead of crashing		

ID	Scenario	Procedure	Robust expected behaviour	Actual result	Status
EC-02	Duplicate return	Return the same issue record twice	Second return is refused; availability is not double-counted		
EC-03	Member with active issue	Delete a member who has a book issued	Deletion blocked, or the user is warned		
EC-04	Issued book deletion	Delete a book that is currently issued	Deletion blocked, or the user is warned		
EC-05	Two tabs	Edit data in two tabs, then reload both	Behaviour documented (last write wins unless synchronised)		
EC-06	Long input	Enter a very long title	Layout remains usable; value stored correctly		
EC-07	Different origin	Compare data at ports 5173, 8080 and 8081	Independent datasets (expected)		

Appendix C: Important Commands

Table C.1: Command reference

Area	Command	Purpose
Development	`npm install`	Install dependencies
Development	`npm run dev`	Start the Vite development server
Development	`npm run build` / `npm run preview`	Create and preview the production build
Git	`git add . && git commit -m "message"`	Record a change
Git	`git push origin main`	Publish commits to GitHub
Docker	`docker build -t library-management-system:1.0 .`	Build the image
Docker	`docker run -d --name library-management-docker -p 8080:80 library-management-system:1.0`	Run the container
Docker	`docker ps`, `docker logs library-management-docker`	Check status and logs
Docker	`docker stop library-management-docker && docker rm library-management-docker`	Remove the container
Kubernetes	`kubectl config use-context docker-desktop`	Select the local cluster
Kubernetes	`kubectl apply -f ./k8s/`	Create or update resources
Kubernetes	`kubectl get all`	Show Deployment, ReplicaSet, Pods and Service
Kubernetes	`kubectl describe pod <name>`, `kubectl logs <name>`	Diagnose a Pod
Kubernetes	`kubectl port-forward service/library-management-service 8081:80`	Open the application locally
Kubernetes	`kubectl rollout restart deployment/library-management-deployment`	Restart Pods after rebuilding the image

Area	Command	Purpose
Kubernetes	`kubectl delete -f ./k8s/`	Remove the resources

Appendix D: Sample Configuration Files

The following files support the configuration described in Chapter 2. Files marked as proposed belong to the future monitoring extension and are not part of the current deployment.

Listing D.1: .dockerignore

```
node_modules
dist
.git
.env
.env.*
!.env.example
```

Listing D.2: .gitignore (relevant entries)

```
node_modules/
dist/
.env
.env.*
!.env.example
```

Listing D.3: .env.example (committed; contains names only)

```
VITE_GRAFANA_FARO_URL=
VITE_GRAFANA_APP_NAME=library-management-system
VITE_GRAFANA_APP_ENV=development
```

Listing D.4: PROPOSED Grafana data-source provisioning (not deployed)

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
```

Listing D.5: PROPOSED Prometheus alerting rule (not deployed)

```
groups:
  - name: library-service
    rules:
      - alert: LibraryServiceDown
        expr: up{job="library-service"} == 0
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Library service target is not reachable by Prometheus"
```